Hook System

Hook System

Table of Contents

1. Hook-Related File Inventory
2. Hook System Design Goals and Overall Architecture
   - Design Philosophy and Core Design Ideas
   - Design Goals
   - Architecture Layer Diagram
   - Core Execution Patterns
3. HookEvent System in Detail
   - All 26 Hook Events
   - Always-Emitted Events
   - HookContext (Event Context)
4. Hook Types and Data Structures
   - Hook Type Overview
   - Command Hook
   - Prompt Hook
   - Agent Hook
   - HTTP Hook
   - Callback Hook (Internal)
5. Hook Output Protocol (JSON Schema)
   - Top-Level Protocol Structure
   - hookSpecificOutput: Event-Specific Output
   - Zod Schema Validation Flow
6. Core Execution Engine
   - executeHooks() Generator Function
   - Execution Flow in Detail
   - Timeout Control
   - HookResult Aggregation
7. Command Hook Executor in Detail
   - Full Execution Flow
   - Key Design of stdin/stdout Exchange Protocol
   - EPIPE Error Handling
   - Async Detection and executeInBackground() Interaction
   - Prompt Request Serialization
8. Prompt Hook Executor in Detail
   - Execution Flow
   - Key Features
   - Compile-Time Type Consistency Assertion
9. Agent Hook Executor in Detail
   - Execution Flow
   - Mechanism for Enforcing StructuredOutput
   - Three-Tier Signal Cancellation Management
   - Dynamic alwaysAllow Rule Injection
   - Prompt Hook vs Agent Hook Comparison
10. HTTP Hook Executor in Detail
    - Execution Flow
    - Environment Variable Interpolation Algorithm
    - Precise Semantics of CRLF Sanitization
    - Precise Semantics of SSRF Protection
11. Callback and Function Hooks
    - Callback Hook
    - Function Hook
12. Async Hook Mechanism and AsyncHookRegistry
    - Two Async Modes
    - AsyncHookRegistry Core Logic
    - Special Handling of SessionStart Async Hooks
    - Cleanup and Finalization
13. Session Hooks: Session-Scoped Hooks
    - SessionHooksState Data Structure
    - Core APIs
    - Deduplication Mechanism
14. Plugin Hooks and Hot Reload
    - Plugin Hook Loading Flow
    - Special Variables for Plugin Hooks
    - Hot Reload Mechanism
    - Cleanup When Plugin is Disabled
15. Matcher System and Conditional Filtering
    - Matcher Concept
    - matchesPattern() Algorithm
    - `if` Conditional Filtering
    - Hook Priority Ordering
16. Hook Execution Event Broadcasting System
    - Three Types of Execution Events
    - Event Filtering Mechanism
    - Pending Event Buffer
    - Progress Push Implementation
17. Key Algorithm Details
    - URL Wildcard Matching
    - Windows POSIX Path Conversion
    - Shell Selection Algorithm
    - Prompt Request Detection and Serialization
    - Multi-Agent Session Isolation
18. Security Mechanisms
    - Multi-Layer Security Overview
    - Trust Level Enumeration
    - Plugin Hook Special Handling
19. Configuration System and Priority
    - Configuration Files and Sources
    - Configuration Merging Rules
    - Same-File Deduplication
20. Snapshot Mechanism
    - Why Snapshots Are Needed
    - Snapshot Lifecycle
    - Relationship with Settings Cache
21. UI Layer Integration
    - `/hooks` CLI Command
    - HooksConfigMenu Component

    - UI Event Flow
22. End-to-End Data Flow Diagram
23. System-Level Design Principles (Meta-Analysis)
24. Appendix: Design Pattern Summary
    - Strategy Pattern
    - Observer Pattern
    - Registry Pattern
    - Pipeline / Generator Pattern
    - Decorator Pattern
    - Anti-Corruption Layer Pattern
25. File Index

---

1. Hook-Related File Inventory

#	File Path	Core Responsibility	Approx. Lines
1	`utils/hooks.ts`	**Core Engine**: executeHooks generator, Command execution, result aggregation	~2700
2	`utils/hooks/AsyncHookRegistry.ts`	Async Hook registry, pending tracking, finalize	~310
3	`utils/hooks/execHttpHook.ts`	HTTP Webhook execution, URL allowlist, SSRF protection	~243
4	`utils/hooks/execPromptHook.ts`	Prompt Hook → Haiku LLM execution	~212
5	`utils/hooks/execAgentHook.ts`	Agent Hook → Subagent multi-turn execution	~340
6	`utils/hooks/hookEvents.ts`	Event broadcasting (started/progress/response)	~193
7	`utils/hooks/hooksSettings.ts`	Hook config reading, priority sorting, display text	~272
8	`utils/hooks/hooksConfigManager.ts`	Hook config CRUD management	~200
9	`utils/hooks/hooksConfigSnapshot.ts`	Startup snapshot capture + ManagedOnly strategy	~134
10	`utils/hooks/sessionHooks.ts`	Session-scoped hook registration, Map storage	~448
11	`types/hooks.ts`	Hook type definitions, Zod Schema, hookSpecificOutput	~291
12	`schemas/hooks.ts`	Zod validation schema for hook config	~200
13	`utils/hooks/hookHelpers.ts`	Helper functions (StructuredOutput / $ARGUMENTS replacement)	~150
14	`utils/plugins/loadPluginHooks.ts`	Plugin hook loading + hot reload	~150
15	`utils/hooks/registerFrontmatterHooks.ts`	Frontmatter hook registration	~100
16	`utils/hooks/registerSkillHooks.ts`	Skill hook registration	~100
17	`components/hooks/HooksConfigMenu.tsx`	React Hook config menu component	~300
18	`commands/hooks/hooks.tsx`	`/hooks` command implementation	~400
19	`entrypoints/sdk/coreTypes.ts`	HOOK_EVENTS constant array definition	~100

---

2. Hook System Design Goals and Overall Architecture

2.0 Design Philosophy and Core Design Ideas

The Hook system is not a simple "event-listener" pattern — it is built around several core constraints:

2.0.1 Security Over Features

A hook is essentially **user-controllable arbitrary code execution** (shell commands, HTTP requests, even custom LLM prompts). This brings enormous security risks. Therefore, the system implements **Defense in Depth**:

```
L1  Trust check (workspace trust)     → Interactive mode requires accepting trust dialog
L2  ManagedOnly strategy              → Enterprise managed environments can force-exclude user-defined hooks
L3  Global disable                    → Admins can completely turn off all hooks
L4  HTTP URL allowlist                → Only pre-approved external endpoints are accessible
L5  SSRF DNS protection               → Blocks requests to private IPs
L6  Header injection protection       → Sanitizes CR/LF/NUL to prevent HTTP header injection
L7  Environment variable allowlist   → $VAR not in allowlist is replaced with empty string
```

Note

Trust checking is performed **centrally at the `executeHooks()` function entry**, not scattered across individual hook executors. This ensures every hook type (command / prompt / agent / http) automatically inherits the security check, preventing new hook types from bypassing security validation.

2.0.2 Snapshot Isolates Config, Policy Controls Layers

Hook configuration is captured as a **snapshot at application startup** via `captureHooksConfigSnapshot()`, and **runtime reads directly from memory**. This solves three problems:

1. **Performance**: Re-reading and parsing settings.json on every hook trigger is extremely expensive (especially for high-frequency `PostToolUse` scenarios)
2. **Consistency**: Ensures hook config stays unchanged within a single session, preventing inconsistent behavior from external edits mid-session
3. **Security isolation**: The snapshot is captured **before** the trust dialog is shown (before any trust state exists), but the trust check (`shouldSkipHookDueToTrust()`) happens **at invocation time**. Together, these prevent a historically real vulnerability — where `SessionEnd` / `SubagentStop` hooks were still executed after the user **rejected** the trust dialog

The snapshot's policy layering (`hooksConfigSnapshot.ts` lines 18–53) implements the **principle of least privilege**:

```
Admin disableAllHooks=true         → All disabled (including managed hooks)
Admin allowManagedHooksOnly=true  → Only managed hooks execute
Non-managed settings disableAllHooks=true → Only managed hooks can still run
Default                            → Merge all sources (backward compatible)
```

Core principle: **Non-managed settings cannot disable managed hooks**. This is a key enterprise security guarantee.

Note

PS:
Non-managed settings are options users configure themselves in `~/.claude/settings.json` and similar files; managed settings are read-only policies centrally pushed by admins via MDM or API. Managed hooks are hooks written in managed settings.
The core security guarantee between the two is: non-managed settings cannot disable managed hooks — concretely: `disableAllHooks` in non-managed settings cannot turn off managed hooks; `allowManagedHooksOnly` only takes effect when written in managed settings, and once set to true, user/project-level hooks are completely ignored; even if a user writes these two fields in settings.json, Claude Code only respects the values in `policySettings`, architecturally guaranteeing enterprise security policies cannot be bypassed by end users.

2.0.3 Performance Adapts to Scenario

The Hook system is not a fixed execution path — it dynamically selects the optimal path based on hook type:

- **Pure internal callback hook fast path**: When all hooks in a batch are internal TS callbacks (`sessionFileAccessHooks`, `attributionHooks`), the entire async generator framework, spans, progress, and AbortSignal mechanisms are skipped
- **Lazy JSON serialization**: All hooks in a batch share a single `hookInput` JSON serialization result (`hookInput` is immutable within a batch), avoiding repeated serialization of expensive complex objects
- **Map instead of Record**: Session hooks use `Map<string, SessionStore>` for storage, avoiding O(N²) object copy overhead in `parallel()` high-concurrency workflows (where multiple schema-mode agents register hooks concurrently)

Note

PostToolUse triggers on every invocation, so when a batch contains only pure internal TypeScript callbacks (sessionFileAccessHooks, attributionHooks), those callbacks always return `{}`, never read the signal, and don't need spans/progress — yet the full async generator framework would be pure overhead. Therefore the code short-circuits: if there are user-defined hooks (command/prompt/http/agent), take the slow path and enable full observability; if only internal callbacks, directly loop through and call `callback()`, skipping all infrastructure.

2.0.4 Tolerant of Failures, Strict on Results

Note

The Hook system takes a **tolerant approach** toward hook **execution** errors (parse failures, timeouts, unstructured output) — logged as `non_blocking_error` but never blocking the main flow. But it is **strict about hook-returned decision results**:

- `continue: false` → blocks operation continuation
- `suppressOutput: true` → hides stdout output
- `stopReason` → actively terminates model generation

This ensures hooks can safely experiment (errors don't affect the main flow) while having the authority to make real critical decisions (blocking dangerous operations).

2.1 Design Goals

The core design goal of the Hook system: **At every key point in Claude Code's lifecycle, allow users (developers) to insert custom logic to change program behavior or integrate with external systems.**

Specific capabilities:

Capability	Description
Tool Interception	Intervene before/after tool execution, validate params, modify input, intercept execution
Permission Control	Custom permission decision logic (allow/deny/ask)
LLM Validation	Use Haiku small model for lightweight tool call judgments
Agent Validation	Use full Subagent for complex multi-turn validation
Webhook Integration	Send structured event notifications to external URLs
Async Execution	Run long tasks in background, wake model on completion
Context Injection	Inject additional context information into the model
Workspace Initialization	SessionStart / Setup Hook for environment preparation
Compaction Coordination	Data processing before/after conversation compaction
MCP Interaction	Elicitation mechanism for MCP server user requests

2.2 Architecture Layer Diagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          Business Callers                                │
│  ToolCoordinator / AgentLoop / SetupHandler / StopHandler                │
│  Call executeHooks({ hookInput, toolUseID, signal, ... })                │
└──────────────────────────────────────────────────────────────────────────┘
                              │ AsyncGenerator<AggregatedHookResult>
                              ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                   executeHooks() Core Generator                          │
│  ┌──────────────────────────────────────────────────────────────────────┐│
│  │ 1. shouldDisableAllHooksIncludingManaged()  → early return           │ │
│  │ 2. CLAUDE_CODE_SIMPLE env var check         → early return           │ │
│  │ 3. shouldSkipHookDueToTrust()              → skip if untrusted      │ │
│  │ 4. getMatchingHooks()  → load & match from snapshot/registry/session│ │
│  │ 5. Internal Hook fast path (execute callbacks directly if no user hooks)│ │
│  │ 6. Beta Tracing: logOTelEvent + startHookSpan                       │ │
│  │ 7. yield progress message (notify UI execution started)             │ │
│  │ 8. Launch hookPromises[] for parallel execution of each hook        │ │
│  └──────────────────────────────────────────────────────────────────────┘│
└──────────────────────────────────────────────────────────────────────────┘
                              │
        ┌─────────────────────┼─────────────────────┐
        ▼                     ▼                     ▼
┌──────────────┐      ┌──────────────┐      ┌──────────────────┐
│ Callback Hook │      │  Prompt Hook  │      │   Agent Hook     │
│ Direct TS     │      │ execPromptHook│      │ execAgentHook    │
│ callback exec │      │ → Haiku LLM   │      │ → Subagent query │
└──────────────┘      └──────────────┘      └──────────────────┘
        │                     │                     │
        └─────────────────────┼─────────────────────┘
                              ▼
┌──────────────────────────────────────────────────────────────────────────┐
│              Command / HTTP Hook Execution Path                          │
│  ┌──────────────────────────────────────────────────────────────────────┐│
│  │ Command: execCommandHook()                                           │ │
│  │   - Shell parsing (bash / PowerShell / Git Bash)                    │ │
│  │   - Env var injection (CLAUDE_PROJECT_DIR / CLAUDE_PLUGIN_ROOT etc.)│ │
│  │   - stdin writes jsonInput                                          │ │
│  │   - stdout streaming → JSON detection (async / sync)                │ │
│  │   - background() → AsyncHookRegistry                                │ │
│  │                                                                     │ │
│  │ HTTP: execHttpHook()                                                │ │
│  │   - URL allowlist validation → SSRF Guard → axios.post              │ │
│  │   - Env var interpolation → Header injection protection             │ │
│  └──────────────────────────────────────────────────────────────────────│ │
└──────────────────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                   processHookJSONOutput()                                │
│  Parse Hook stdout → Zod validation → map to HookResult                  │
│  Supports 16 hookSpecificOutput event types                              │
└──────────────────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────────────────┐
│              Hook Execution Event Broadcasting (hookEvents.ts)           │
│  HookStartedEvent → HookProgressEvent → HookResponseEvent                │
│  → UI / SDK / Telemetry                                                  │
```

2.3 Core Execution Patterns

The Hook system executes using an **AsyncGenerator** pattern, which provides three advantages:

1. **Progressive output**: When a hook starts executing, it immediately yields a progress message, allowing the UI to display status in real time
2. **Parallel execution**: All hooks are launched in parallel via `Promise.allSettled`
3. **Streaming processing**: Command Hook stdout can push progress events in real time

> Note:
> The "parallel" here refers to parallel execution of multiple hooks triggered by the same event (Promise.allSettled runs hook A and hook B in PreToolUse simultaneously), not parallel execution of hooks and the LLM main flow.
> In fact, hooks are blocking to the LLM main flow — PreToolUse must wait for all hooks to finish before the tool actually executes, and PostToolUse also waits for hooks to yield before continuing. The design does not run hooks in the background non-blockingly chasing LLM API calls.

---

3. HookEvent System in Detail

3.1 All 26 Hook Events

Defined in: `restored-src/src/entrypoints/sdk/coreTypes.ts` (`HOOK_EVENTS` constant)

#	Event Name	Trigger Timing	`matchQuery` Source	Typical Use
1	`PreToolUse`	Before tool execution	`tool_name`	Param validation, permission check, logging
2	`PostToolUse`	After tool succeeds	`tool_name`	Result processing, side effects
3	`PostToolUseFailure`	After tool fails	`tool_name`	Error handling
4	`PermissionRequest`	When permission dialog is shown	`tool_name`	Custom permission decisions
5	`PermissionDenied`	When auto-classifier denies tool	`tool_name`	Denial analysis, retry decisions
6	`Notification`	When sending notification	`notification_type`	Notification interception/enhancement
7	`UserPromptSubmit`	When user submits prompt	none	Prompt preprocessing
8	`SessionStart`	When new session starts	`source`	Environment initialization
9	`SessionEnd`	When session ends	`reason`	Cleanup/summarization
10	`Stop`	Before Claude finishes response	none	Final processing
11	`StopFailure`	When API error causes stop	`error`	Error diagnostics
12	`SubagentStart`	When Subagent starts	`agent_type`	Subagent tracking
13	`SubagentStop`	When Subagent stops	`agent_type`	Subagent result processing
14	`PreCompact`	Before conversation compaction	`trigger`	Pre-compaction preparation
15	`PostCompact`	After conversation compaction	`trigger`	Post-compaction verification
16	`Setup`	When repo setup Hook runs	`trigger`	Environment setup
17	`TeammateIdle`	When teammate is idle	none	Coordination
18	`TaskCreated`	When task is created	none	Task tracking
19	`TaskCompleted`	When task completes	none	State update
20	`Elicitation`	When MCP server requests user input	`mcp_server_name`	MCP interaction handling
21	`ElicitationResult`	When user responds to Elicitation	`mcp_server_name`	Result processing
22	`ConfigChange`	When config file changes	`source`	Config response
23	`WorktreeCreate`	When isolated Worktree is created	none	Environment preparation
24	`WorktreeRemove`	When Worktree is removed	none	Cleanup
25	`InstructionsLoaded`	When instruction file loads	`load_reason`	Instruction preprocessing
26	`CwdChanged`	When working directory changes	none	Path-related processing
27	`FileChanged`	When watched file changes	`basename(file_path)`	File change response

> Note: `FileChanged` was added after `CwdChanged`.

3.2 Always-Emitted Events

Defined in: `restored-src/src/utils/hooks/hookEvents.ts` (line 18)

```
const ALWAYS_EMITTED_HOOK_EVENTS = ['SessionStart', 'Setup'] as const
```

These two events are **always broadcast to the UI**, regardless of the `includeHookEvents` option, so users can always see logs for **session startup and repo setup**.

3.3 HookContext (Event Context)

The `HookInput` structure passed on each event trigger (example: `PreToolUseHookInput`):

```
// Source: restored-src/src/entrypoints/sdk/coreSchemas.ts:414
interface PreToolUseHookInput {
  hook_event_name: 'PreToolUse'
  session_id: string
  transcript_path: string
  cwd: string
  permission_mode?: string
  agent_id?: string         // Only set when triggered by Subagent; not set on main thread (even in --agent sessions)
  agent_type?: string       // Subagent type (e.g. "general-purpose"); also set on main thread in --agent sessions
  tool_name: string
  tool_input: Record<string, unknown>
  tool_use_id: string
}
```

Common base fields (constructed via `createBaseHookInput()`, hooks.ts:301):

```
// Source: restored-src/src/utils/hooks.ts:301
{
  session_id: string        // Session ID
  transcript_path: string   // Transcript file path
  cwd: string               // Current working directory
  permission_mode?: string  // Permission mode
  agent_id?: string         // Agent ID (distinguishes Subagent from main thread)
  agent_type?: string       // Agent type
}
```

---

4. Hook Types and Data Structures

4.1 Hook Type Overview

Defined in: `restored-src/src/utils/settings/types.ts`

The hook config union type is `HookCommand`, with five concrete types:

```
type HookCommand =
  | CommandHook       // Shell command
  | PromptHook        // LLM prompt evaluation
  | AgentHook         // Multi-turn agent validation
  | HttpHook          // HTTP Webhook
  | CallbackHook      // Internal TypeScript callback
```

There is also `FunctionHook` (Session-scoped TypeScript callback) that cannot be persisted to `settings.json`.

4.2 Command Hook

Command Hook is the most commonly used hook type, executing a shell command (bash or powershell) in a subprocess.

```
// Source: restored-src/src/schemas/hooks.ts:32 (BashCommandHookSchema)
interface CommandHook {
  type: 'command'              // Identifies as command type
  command: string               // Shell command to execute, supports ${variable} interpolation
  if?: string                   // Conditional expression (permission rule syntax, e.g. "Bash(git *)")
                                // Filtered by tool_name/tool_input before subprocess spawn; skipped if no match
  shell?: 'bash' | 'powershell' // Shell type to execute:
                                // 'bash' uses $SHELL (bash/zsh/sh); 'powershell' uses pwsh
                                // Default: bash
  timeout?: number              // Timeout in seconds for this command (positive integer)
                                // Note: not default 600s; framework-level default timeout is controlled separately at call site
  statusMessage?: string        // Custom status text shown in spinner during hook execution
  once?: boolean                // true=automatically removed from config after one execution (common for init hooks)
  async?: boolean               // true=background async execution, doesn't block model; continues immediately after return
  asyncRewake?: boolean         // true=background execution, wakes model if exit code is 2 (blocking error); implies async: true
}
```

Note

A Command Hook is essentially **a shell command that runs automatically at a checkpoint**; external systems don't need to know Claude Code exists — the shell command itself decides what to do.

Typical uses:

- Auto-formatting: PostToolUse + prettier auto-formats code after every Write/Edit
- Pre-commit checks: PreToolUse + git commitlint runs lint checks before commits
- File change notifications: PostToolUse + inotifywait listens for file changes to trigger follow-up flows
- Environment initialization: SessionStart + environment check script ensures dev environment is ready
- Execution result determines subsequent behavior via exit code: **0** = continue normal flow, **2** = block tool execution, other = display output only without blocking
- Matcher matches tool names, `if` condition filters tool input; mismatches cause skip (no subprocess spawned)

4.3 Prompt Hook

A Prompt Hook uses an LLM to make intelligent judgments about tool calls (commonly used for PreToolUse security review, PostToolUse result validation).

```
// Source: restored-src/src/schemas/hooks.ts:67 (PromptHookSchema)
interface PromptHook {
  type: 'prompt'
  prompt: string               // Prompt sent to LLM; $ARGUMENTS placeholder is replaced with full JSON of tool call
  if?: string                  // Conditional filter (same as Command Hook, permission rule syntax)
  timeout?: number             // Timeout in seconds (default 30s)
  statusMessage?: string       // Status text shown in spinner
  once?: boolean               // true=automatically removed after one execution
  model?: string               // Optional custom model; if omitted, defaults to Haiku (faster and cheaper than main model)
}
```

Applicable events: `PreToolUse` (can block tool execution on exit code 2), `PostToolUse` (checks if results match expectations), `PermissionRequest` (judges whether to allow). Other events don't support Prompt Hook. LLM return content flows into `hookSpecificOutput` to affect subsequent processes.

Note

A Prompt Hook is essentially **an LLM call at a checkpoint to make a judgment** — a micro LLM inference without tool capabilities.

Typical uses:

- Security review: PreToolUse + LLM judges if this Write operation might contain sensitive info
- Result validation: PostToolUse + LLM checks if git diff meets code standards
- Permission judgment: PermissionRequest + LLM decides whether to allow based on context
- LLM input is the full JSON of the tool call (injected via `$ARGUMENTS` placeholder), returns `{ok: true/false, reason?: string}`
- Single call (30s timeout), lighter than Agent Hook, suitable for quick judgments without needing tools
- Execution entry: `execPromptHook` (`restored-src/src/utils/hooks/execPromptHook.ts`)

4.4 Agent Hook

An Agent Hook launches a complete **multi-turn Agent** (with tool access) to perform complex validation on tool calls (e.g., running tests, checking results). The key difference from Prompt Hook: Prompt Hook is a single LLM call without tools; Agent Hook is a full agent loop (up to 50 turns), autonomously calling tools for deep validation.

```
// Source: restored-src/src/schemas/hooks.ts:128 (AgentHookSchema)
interface AgentHook {
  type: 'agent'
  prompt: string               // Agent prompt; $ARGUMENTS placeholder is replaced with full JSON of tool call
                               // Example: "Verify that unit tests ran and passed."
  if?: string                 // Conditional filter (same as Command Hook)
  timeout?: number            // Timeout in seconds (default 60s)
  statusMessage?: string       // Status text shown in spinner
  once?: boolean               // true=automatically removed after one execution
  model?: string               // Optional custom model; if omitted, defaults to Haiku (lightweight fast)
}
```

Applicable events: `PreToolUse`, `PostToolUse`, `PermissionRequest`, `Stop`, etc. The agent has a full toolset (default max 50 turns), can read transcript files for context analysis, and its return value flows into `hookSpecificOutput` to affect subsequent processes. Its return value must be output via the StructuredOutput tool as `{ok: boolean, reason?: string}`, otherwise it's treated as cancelled.

Note

- Key differences between Agent Hook and Prompt Hook:
  - **Prompt Hook**: Single LLM call (30s timeout), no tool use, lightweight judgment, more oriented toward the LLM itself
  - **Agent Hook**: Multi-turn agent loop (60s timeout, max 50 turns), full toolset available, can do cross-validation, more oriented toward the Agent
- Typical scenarios: `PostToolUse` + agent hook starts an agent after file write to run tests and validate output; `Stop` event + agent hook validates overall execution results at session end ("did all tests pass?")
- Agent Hook's toolset filters out `ALL_AGENT_DISALLOWED_TOOLS` (prevents creating sub-agents, prevents entering Plan Mode), preventing infinite recursion in stop hook
- Execution entry: `execAgentHook` (`restored-src/src/utils/hooks/execAgentHook.ts`)

4.5 HTTP Hook

An HTTP Hook sends a POST request to an external service (carrying the complete hook input JSON as the body), for integrating CI/CD systems, ticketing platforms, etc. Compared to Command Hook, HTTP Hook is better suited for scenarios requiring interaction with remote services.

```
// Source: restored-src/src/schemas/hooks.ts:97 (HttpHookSchema)
interface HttpHook {
  type: 'http'
  url: string                  // Target URL (must be a valid URL)
  if?: string                  // Conditional filter (same as Command Hook)
  timeout?: number             // Timeout in seconds
  headers?: Record<string, string>   // Extra request headers; values can contain $VAR_NAME or ${VAR_NAME} variable references
                                     // Example: "Authorization": "Bearer $MY_TOKEN"
                                     // Only variables listed in allowedEnvVars are expanded
  allowedEnvVars?: string[]    // List of env var names allowed for interpolation in header values
                                // $VAR not in the list stays as-is (not expanded to empty string)
  statusMessage?: string       // Status text shown in spinner
  once?: boolean               // true=automatically removed after one execution
}
```

Applicable events: All events can use HTTP Hook; typical scenarios include `SessionEnd` (notify external on session end), `Stop` (trigger CI flow on task completion), `PostToolUse` (notify external system after tool execution), etc.

Note

An HTTP Hook is essentially **pushing event data to an external URL** (POST JSON body), letting external systems be aware of what happened inside Claude Code.

Typical uses:

- Notify external systems: POST to Slack, PagerDuty, Feishu on task completion
- Trigger CI/CD: Notify deployment platform to start building after code write
- Audit logging: Log every tool call to an external logging system
- Sync status to tickets: Auto-update Jira/Linear ticket status on completion

4.6 Callback Hook (Internal)

A Callback Hook is a **hook callback registered directly as a TypeScript function in code** — not a config entry in `settings.json`, but logic dynamically registered by the framework itself or by plugins. It is fundamentally different from Command/Prompt/Agent/HTTP Hooks — the latter are externally configurable, while the former are programmatically embedded.

```
// Source: restored-src/src/types/hooks.ts:211 (HookCallback)
interface HookCallback {
  type: 'callback'
  callback: (
    input: HookInput,                   // Event input
    toolUseID: string | null,           // Tool call ID (null for SessionStart etc.)
    abort: AbortSignal | undefined,      // Can listen for abort signals
    hookIndex?: number,                  // Used to compute CLAUDE_ENV_FILE path on SessionStart
    context?: HookCallbackContext,       // State access context (contains getAppState, updateAttributionState)
  ) => Promise<HookJSONOutput>
  timeout?: number
  internal?: boolean      // true=internal hook (e.g. sessionFileAccessHooks, attributionHooks)
                          // internal hooks are included in the pure-internal-callback fast path (hooks.ts:2037), bypassing subsequent observability framework
}
```

Typical representatives: `sessionFileAccessHooks` (file access permission checking) and `attributionHooks` (commit attribution). `internal: true` hooks are included in fast path optimization (`isInternalHook` logic in `hooks.ts:1440`), bypassing the subsequent observability framework, returning `{}` directly with no span/progress.

Note

A Callback Hook is essentially **a hook function written by the framework itself**, registered at a lifecycle checkpoint to perform specific functions.

Typical uses:

- File access tracking: Records file access on every Read tool call to the session (`sessionFileAccessHooks`)
- Commit attribution: Injects CLAUDE_ENV_FILE path at SessionStart for attribution use (`attributionHooks`)
- Plugin extension: Plugins register their own callback logic via the `useMasterHandler` API
- Since it's called directly by code, there is no stdout/stderr output; the framework reads the function return value to determine subsequent behavior
- `internal: true` hooks are optimized into the fast path to avoid unnecessary observability overhead

---

5. Hook Output Protocol (JSON Schema)

5.1 Top-Level Protocol Structure

Defined in: `restored-src/src/types/hooks.ts` (lines 50–176)

Hooks must return JSON via stdout in one of two forms:

```
// Async response (immediate return, hook runs in background)
{ "async": true, "asyncTimeout": 15000 }

// Sync response (full result)
{
  "continue"?: boolean,           // Whether to continue (default true)
  "suppressOutput"?: boolean,     // Whether to hide stdout
  "stopReason"?: string,          // Stop reason
  "decision"?: 'approve' | 'block', // Permission decision (legacy)
  "reason"?: string,               // Decision reason
  "systemMessage"?: string,        // System message
  "hookSpecificOutput": { ... }   // Event-specific output
}
```

5.2 hookSpecificOutput: Event-Specific Output

Each event has its own supported output fields:

PreToolUse Event

```
{
  "hookEventName": "PreToolUse",
  "permissionDecision": "allow" | "deny" | "ask",
  "permissionDecisionReason": "string",
  "updatedInput": { "key": "value" },   // Modified tool parameters
  "additionalContext": "string"          // Injected additional context
}
```

UserPromptSubmit Event

```
{ "hookEventName": "UserPromptSubmit", "additionalContext": "string" }
```

SessionStart Event

```
{
  "hookEventName": "SessionStart",
  "additionalContext": "string",
  "initialUserMessage": "string",
  "watchPaths": ["/abs/path1", "/abs/path2"]
}
```

PostToolUse Event

```
{
  "hookEventName": "PostToolUse",
  "additionalContext": "string",
  "updatedMCPToolOutput": unknown
}
```

PermissionDenied Event

```
{ "hookEventName": "PermissionDenied", "retry": boolean }
```

PermissionRequest Event

```
{
  "hookEventName": "PermissionRequest",
  "decision": {
    "behavior": "allow" | "deny",
    "updatedInput"?: object,
    "updatedPermissions"?: PermissionUpdate[]
  }
}
```

Elicitation / ElicitationResult Events

```
{
  "hookEventName": "Elicitation",
  "action": "accept" | "decline" | "cancel",
  "content": { "key": "value" }
}
```

WorktreeCreate / CwdChanged / FileChanged Events

```
{
  "hookEventName": "WorktreeCreate",
  "worktreePath": "/abs/path"
}
```

Note

The relationship between Events and Hook types: They are a **two-dimensional orthogonal design**.

- **Event** = WHEN (when to trigger): The framework buries 22 checkpoints in the main loop (PreToolUse, PostToolUse, SessionEnd, etc.), defining when hooks trigger
- **Hook Type (Command / Prompt / Agent / HTTP)** = WHAT (how to execute): Defines what logic executes when a checkpoint fires

Their relationship can be understood like this: in `settings.json`, the event name is the first-level key, defining "where to hang it"; the hook type is the `type` field, defining "what logic to hang". The same event can have multiple hook types (executed in order), and the same hook type can appear in different events.

Example:

```
{
  "PreToolUse": [{                        ← Event = WHEN (when to trigger)
    "matcher": "Bash",
    "hooks": [
      { "type": "command", "command": "..." },  ← all four types supported
      { "type": "prompt",  "prompt": "..." },
      { "type": "agent",   "prompt": "..." },
      { "type": "http",    "url": "..." }
    ]
  }]
}
```

5.3 Zod Schema Validation Flow

```
// Step 1: Check if async response
isAsyncHookJSONOutput(parsed)  // 'async' in parsed && parsed.async === true

// Step 2: Sync validation
hookJSONOutputSchema().safeParse(parsed)
```

```
// where syncHookResponseSchema() contains the full hookSpecificOutput union type

// Step 3: Format errors on validation failure
validation.error.issues.map(
  err => `  - ${err.path.join('.')}: ${err.message}`
)
```

---

6. Core Execution Engine

6.1 executeHooks() Generator Function

Defined in: `restored-src/src/utils/hooks.ts` (lines 1952–2600+)

This is the **core engine** of the Hook system, implemented as an **AsyncGenerator** that allows step-by-step yielding of execution results.

```
async function* executeHooks({
  hookInput,           // Event input (PreToolUseHookInput etc.)
  toolUseID,           // Tool call ID
  matchQuery,          // Query string
  signal,              // AbortSignal (cancellable)
  timeoutMs,           // Default 600000ms
  toolUseContext,      // Tool context (contains tools / agentId / setAppState)
  messages,            // Conversation history (needed by Agent/Prompt Hook)
  forceSyncExecution,  // Force sync execution
  requestPrompt,       // Prompt request callback within hook
  toolInputSummary,    // Tool input summary (for UI display)
}): AsyncGenerator<AggregatedHookResult>
```

6.2 Execution Flow in Detail

Stage 1: Guard Checks

```
// 1.1 Managed policy: completely disable all hooks
if (shouldDisableAllHooksIncludingManaged()) return

// 1.2 Simple mode: skip all hooks
if (isEnvTruthy(process.env.CLAUDE_CODE_SIMPLE)) return

// 1.3 Trust check: interactive mode requires workspace trust for all hooks
if (shouldSkipHookDueToTrust()) return
```

> **Important security design**: All hooks in interactive mode require `checkHasTrustDialogAccepted()` to return true, even if the hook config was captured by `captureHooksConfigSnapshot()` **before** the trust dialog was shown. This prevents a historically real vulnerability where SessionEnd/SubagentStop hooks still executed after the user rejected trust.

Stage 2: Load Matching Hooks

```
const appState = toolUseContext?.getAppState?.() ?? undefined
const sessionId = toolUseContext?.agentId ?? getSessionId()
const matchingHooks = await getMatchingHooks(
  appState,
  sessionId,
  hookEvent,
  hookInput,
  toolUseContext?.options?.tools,
)
```

`getMatchingHooks()` does three things internally:

1. Load configured hooks from snapshot via `getHooksConfigFromSnapshot()`
2. Load SDK-registered hooks from `getRegisteredHooks()`
3. Load Session hooks from `getSessionHooks()` / `getSessionFunctionHooks()`

Stage 3: Fast Path (Pure Callback Hook)

```
// If all are internal callback hooks, take fast path (-70% overhead)
if (matchingHooks.every(m => m.hook.type === 'callback')) {
  const batchStartTime = Date.now()
  for (const [i, { hook }] of matchingHooks.entries()) {
    await hook.callback(hookInput, toolUseID, signal, i, context)
  }
  getStatsStore()?.observe('hook_duration_ms', Date.now() - batchStartTime)
  return
}
```

Why is this safe? Internal callback hooks (like `sessionFileAccessHooks`, `attributionHooks`) return `{}` and don't use abort signals, so the skipped mechanisms (span, progress, AbortSignal, processHookJSONOutput, resultLoop) are genuinely unnecessary.

Stage 3.5: Lazy JSON Serialization

```
// hooks.ts:2121-2140
// Lazy-once stringify of hookInput. Shared across all command/prompt/agent/http
// hooks in this batch (hookInput is never mutated). Callback/function hooks
// return before reaching this, so batches with only those pay no stringify cost.
let jsonInputResult: ...
function getJsonInput() {
  if (jsonInputResult !== undefined) return jsonInputResult
  try {
    return (jsonInputResult = { ok: true, value: jsonStringify(hookInput) })
  } catch (error) {
    logError(Error(`Failed to stringify hook ${hookName} input`, { cause: error }))
    return (jsonInputResult = { ok: false, error })
  }
}
```

This is a **lazy-once** pattern: serialization only happens when hook executors actually need the JSON input, and the result is shared across the batch. Internal callback/function hooks return before reaching this code, so they pay zero serialization cost.

Stage 4: Beta Tracing Instrumentation

This is the **observability instrumentation** of the Hook system, only triggered when Beta Tracing is enabled. OpenTelemetry (OTEL) records full-chain tracing data for hook execution, including execution start events and span durations.

```
if (isBetaTracingEnabled()) {
  void logOTelEvent('hook_execution_start', {
    hook_event: hookEvent,
    hook_name: hookName,
    num_hooks: String(matchingHooks.length),
    managed_only: String(shouldAllowManagedHooksOnly()),
    hook_definitions: jsonStringify(getHookDefinitionsForTelemetry(...)),
  })
  const hookSpan = startHookSpan(hookEvent, hookName, matchingHooks.length, ...)
}
```

Recorded content includes: `hook_event` (event name, e.g. PreToolUse), `hook_name` (full hook name, e.g. PreToolUse:Write), `num_hooks` (matched hook count), `managed_only` (whether managed mode), `hook_definitions` (hook definitions). The corresponding execution-end event `hook_execution_complete` is recorded after all hooks finish, including counts for success/blocking/non_blocking_error/cancelled. The Span created by `startHookSpan` is closed by `endHookSpan` at the end of Stage 7, forming a complete tracing chain.

Stage 5: Parallel Execution of All Hooks

```
const hookPromises = matchingHooks.map(async function* ({ hook, pluginRoot, pluginId, skillRoot }, hookIndex) {
  // Launch one async generator per hook
  if (hook.type === 'callback')    → executeHookCallback(...)
  if (hook.type === 'function')    → executeFunctionHook(...)
  if (hook.type === 'prompt')      → execPromptHook(...)
  if (hook.type === 'agent')       → execAgentHook(...)
  if (hook.type === 'http')        → execHttpHook(...)
  if (hook.type === 'command')     → execCommandHook(...)
})
```

6.3 Timeout Control

```
const TOOL_HOOK_EXECUTION_TIMEOUT_MS = 10 * 60 * 1000  // 10 minutes

// SessionEnd Hook has a stricter timeout
const SESSION_END_HOOK_TIMEOUT_MS_DEFAULT = 1500  // 1.5 seconds
// Can be overridden via CLAUDE_CODE_SESSIONEND_HOOKS_TIMEOUT_MS env var
```

Note

Why is SessionEnd's timeout only 1.5 seconds?
SessionEnd is the cleanup phase when a session ends. Hooks' goal is to do summarization, reporting, and other wrap-up work — they cannot affect the user's actual exit experience. 1.5 seconds is a balance between user experience and feature completeness — enough for lightweight reporting, but won't make the user noticeably feel "stuck on exit". If a hook needs to run time-consuming operations, it should declare itself as async with `async: true`.

6.4 HookResult Aggregation

Defined in: `restored-src/src/utils/hooks.ts`

`AggregatedHookResult` is the aggregated result yielded by `executeHooks`:

```
type AggregatedHookResult = {
  message?: Message                        // Attachment message
  blockingErrors?: HookBlockingError[]      // Multiple blocking errors
  preventContinuation?: boolean             // Prevent continuation
  stopReason?: string                       // Stop reason
  hookPermissionDecisionReason?: string      // Permission decision reason
  permissionBehavior?: PermissionResult['behavior']  // 'allow' | 'deny' | 'ask' | 'passthrough'
  additionalContexts?: string[]              // Multiple additional contexts
  initialUserMessage?: string                // SessionStart-specific
  updatedInput?: Record<string, unknown>    // Modified tool parameters
  updatedMCPToolOutput?: unknown             // MCP tool output update
  permissionRequestResult?: PermissionRequestResult  // Permission request result
  watchPaths?: string[]                      // File watch paths
  elicitationResponse?: ElicitResult         // MCP Elicitation response
  elicitationResultResponse?: ElicitResult
  retry?: boolean                            // PermissionDenied retry
}
```

Note

`AggregatedHookResult` is the aggregated summary result after all hooks finish executing. Its final destinations fall into three categories:

1. Tool execution flow control (most critical): Consumed by `runPreToolUseHooks` / `runPostToolUseHooks` in `toolHooks.ts`; processing logic in the switch statement in `toolExecution.ts`:

- `hookPermissionResult` → controls whether tool is allowed (allow / deny / ask), enters permission decision flow
- `hookUpdatedInput` → replaces tool parameters; tool actually uses hook-modified input
- `preventContinuation` → prevents Agent from continuing, may trigger stopReason
- `updatedMCPToolOutput` → replaces MCP tool return result

2. UI message rendering: The `message` field (attachment message) is directly pushed to `resultingMessages[]` returned to the frontend, displaying hook execution results (progress, errors, success, etc.)

3. Observability: Outcome counts (success / blocking / non_blocking_error / cancelled) are recorded by `endHookSpan` to OTEL, forming a complete tracing chain

Simply put: **Hook results ultimately decide two things — whether the tool executes (permissions + input), and what the user sees (messages).**

---

7. Command Hook Executor in Detail

7.1 Full Execution Flow

Defined in: `restored-src/src/utils/hooks.ts` `execCommandHook()` function (~600 lines)

```
┌─────────────────────────────────────────────────────────┐
│ Step 1: Shell Selection                                 │
│  hook.shell → 'bash' (Windows: Git Bash)                │
│  hook.shell → 'powershell' → pwsh -NoProfile -Command |
└─────────────────────────────────────────────────────────┘
                              │
                              ▼
```

```
┌──────────────────────────────────────────────────────────┐
│ Step 2: Windows POSIX Path Conversion                    │
│  Windows non-PowerShell: windowsPathToPosixPath()        │
│  C:\Users\foo → /c/Users/foo                             │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│ Step 3: Variable Substitution                            │
│  ${CLAUDE_PLUGIN_ROOT}  → plugin root directory          │
│  ${CLAUDE_PLUGIN_DATA}  → plugin data directory          │
│  ${user_config.X}       → plugin user config             │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│ Step 4: Windows .sh Auto-prepend 'bash '                 │
│  bash mode only; PowerShell skips this                   │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│ Step 5: SHELL_PREFIX Wrapping (bash mode)                │
│  CLAUDE_CODE_SHELL_PREFIX → formatShellPrefixCommand()   │
│  PowerShell skips (not supported)                        │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│ Step 6: Environment Variable Construction                │
│  subprocessEnv() + CLAUDE_PROJECT_DIR                    │
│  CLAUDE_PLUGIN_ROOT / CLAUDE_PLUGIN_DATA                 │
│  CLAUDE_PLUGIN_OPTION_* (plugin config)                  │
│  CLAUDE_ENV_FILE (SessionStart/Setup/CwdChanged only)    │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│ Step 7: Working Directory Security Check                 │
│  hookCwd = getCwd()                                      │
│  safeCwd = pathExists(hookCwd) ? hookCwd : getOriginalCwd()│
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│ Step 8: spawn Subprocess                                 │
│  Bash: spawn(cmd, [], { shell: gitBashPath ?? true })    │
│  PS:   spawn(pwshPath, ['-NoProfile','-NonInteractive',  │
│                         '-Command', cmd], {...})         │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│ Step 9: stdin Writes JSON Input                          │
│  child.stdin.write(jsonInput + '\n', 'utf8')             │
│  child.stdin.end()                                       │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│ Step 10: stdout/stderr Streaming + Real-time Event Push  │
│  HookProgressEvent (pushed every 1000ms)                 │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│ Step 11: First-line JSON Detection (async vs sync)       │
│  {"async": true, ...}  → background() → AsyncHookRegistry│
│  Other JSON / non-JSON → wait for 'close' event          │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│ Step 12: processExit                                     │
│  exitCode=0 → outcome='success'                          │
│  exitCode=2 (asyncRewake) → enqueueNotification()        │
│  Other non-zero → outcome='non_blocking_error'           │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│ Step 13: stdout → JSON Parse → HookResult                │
│  parseHookOutput(stdout) → processHookJSONOutput()       │
└──────────────────────────────────────────────────────────┘
```

7.2 Key Design of stdin/stdout Exchange Protocol

7.2.1 stdin Write Timing and EOF Behavior

Hook commands receive JSON input via **stdin single-line write**:

```
// hooks.ts L1210 — trailing '\n' is mandatory
child.stdin.write(jsonInput + '\n', 'utf8')
if (!requestPrompt) {
  child.stdin.end()
}
```

Why is `\n` mandatory? The source comment (L1001-1005) explains the critical reason:

> bash `read -r line` returns exit 1 at EOF (no newline), causing the branch to be skipped. With `\n` present, behavior is correct. Therefore both the config-style async hook and the sync path must be consistent.

Meaning: if the hook command uses `read -r line` to read config from stdin, without `\n` bash considers EOF to arrive prematurely, causing the read to fail.

7.2.2 Two-Phase stdout Processing

Hook stdout processing is divided into two strict phases:

Phase 1: Async First-line Detection (`initialResponseChecked`)

```
// hooks.ts L1112-1164
child.stdout.on('data', data => {
  stdout += data
  if (!initialResponseChecked) {
    const firstLine = firstLineOf(stdout).trim()
    if (!firstLine.includes('}')) return  // Haven't received complete JSON line yet
    initialResponseChecked = true
```

```
    const parsed = jsonParse(firstLine)
    if (isAsyncHookJSONOutput(parsed) && !forceSyncExecution) {
      executeInBackground({ processId, ... })  // Immediately background
    }
  }
})
```

Key point: Only the **first line** is checked. If the process quickly outputs multiple lines of data, only the first line is still parsed. This is intentional — a fast-completing hook (e.g. `echo '{"async":true}'; some_command`) would have accumulated multiple lines in `stdout`, and parsing from the entire content would miss the first line, causing async detection to fail and blocking until timeout.

Note

"Only checking the first line" is a trade-off between throughput and correctness
Async detection must respond immediately after the process quickly returns `{"async":true}` (not waiting for subsequent commands to finish), while also being compatible with the scenario of "same command both declares async and outputs real content." Solution: use only the first line as the detection basis; all subsequent output is treated as "normal stdout." This means if a hook outputs malformed format (e.g. multi-line JSON like `{"async": true\n} {"ok": true}`), only the first line is treated as the async declaration and the rest is treated as ordinary stdout — an acceptable trade-off.

Phase 2: Wait for Stream Close (`stdoutEndPromise` / `stderrEndPromise`)

```
// hooks.ts L1234-1260
child.on('close', code => {
  // Wait for all 'data' event handlers to finish (i.e. stdout.end/stderr.end) before resolving
  void Promise.all([stdoutEndPromise, stderrEndPromise]).then(() => {
    const finalStdout = processedPromptLines.size === 0
      ? stdout
      : stdout.split('\n').filter(line => !processedPromptLines.has(line.trim())).join('\n')
    resolve({ stdout: finalStdout, stderr, output, status: exitCode!, aborted: signal.aborted })
  })
})
```

Using `Promise.all([stdoutEndPromise, stderrEndPromise])` ensures that after the process closes, **all buffered data has already been processed by `data` events**, avoiding a race condition where the `close` event fires first but stdout data hasn't been fully read yet.

7.2.3 Dynamic Stripping of Prompt Request Lines

```
// hooks.ts L1243-1249 — fail-closed strategy
const finalStdout =
  processedPromptLines.size === 0
    ? stdout
    : stdout.split('\n').filter(line => !processedPromptLines.has(line.trim())).join('\n')
```

Note

During hook execution, the hook can output `{"type":"prompt",...}` to send an interactive request to Claude Code (e.g., asking the user a question). The framework captures it via the `requestPrompt` callback, handles the response, and writes back to stdin. The entire interaction is completely transparent to the user — **these intermediate prompt request lines are filtered out from stdout after the process ends**. The user ultimately only sees the hook's real output result.

7.2.4 Prompt Hook's stdin Stays Open

Non-prompt-type hooks immediately close stdin, letting the hook command detect EOF right away; prompt-type hooks keep stdin open to receive subsequent responses.

```
// hooks.ts L1211-1214
child.stdin.write(jsonInput + '\n', 'utf8')
if (!requestPrompt) {
  child.stdin.end()  // Non-prompt-type: close stdin
}
// Prompt-type: stdin stays open, waiting for subsequent prompt response writes
```

Note

The open/close of stdin reflects the lifecycle difference between two types of hooks:

- **Non-prompt-type** immediately closes after writing initial input, letting the hook know input is complete and it can start executing
- **Prompt-type** must keep stdin open because it will receive subsequent responses from the framework during execution (via the `processedPromptLines` mechanism), and stdin is only truly closed when all interactions are complete.

7.3 EPIPE Error Handling

When a hook command exits before reading stdin, it triggers an EPIPE error:

```
if (code === 'EPIPE') {
  return { stdout: '', stderr: 'Hook command closed stdin before input written (EPIPE)', status: 1 }
}
```

Note

EPIPE is a friendly fallback for user hook errors
When EPIPE occurs, it means the process exited before reading stdin. This is usually a bug in the user's hook script (e.g. `exit 1` written before `read`). The code doesn't throw an exception; instead, it returns a `non_blocking_error` result with a friendly error message — both informing the user something went wrong and not blocking the main flow. This is a concrete manifestation of the "tolerant failures" principle.

7.4 Async Detection and executeInBackground() Interaction

7.4.1 Two Ways to Trigger Async Background Hooks

There are two paths for hook backgrounding: **known in advance** (config declared) and **discovered at runtime** (runtime detection):

Way	How it's known	When it's processed	Typical scenario
Config-declared	User wrote `"async": true` in `settings.json`	Backgrounded **before** hook starts	Hook author explicitly knows the command is time-consuming
Runtime-detected	Hook outputs first line `{"async": true}`	Discovered **after reading first line** while hook runs	Hook internally decides at runtime whether to go background

```
// Config-declared: background immediately after writing stdin (hooks.ts L995-1008)
if ((hook.async || hook.asyncRewake) && !forceSyncExecution) {
  child.stdin.write(jsonInput + '\n', 'utf8')
  child.stdin.end()
  stdinWritten = true
  const backgrounded = executeInBackground({ processId, ... })
```

```
  if (backgrounded) {
    return { stdout: '', stderr: '', output: '', status: 0, backgrounded: true }
  }
}

// Runtime-detected: background only after stdout has data (hooks.ts L1127-1150)
if (isAsyncHookJSONOutput(parsed) && !forceSyncExecution) {
  const backgrounded = executeInBackground({ processId, ... })
  if (backgrounded) {
    shellCommandTransferred = true
    asyncResolve?.({ stdout, stderr, output, status: 0 })  // resolve immediately
  }
}
```

Essentially this is a **advance commitment vs. deferred report** trade-off — if the user declares it in the config, no guessing is needed; if not declared, the hook itself outputs `{"async": true}` to inform the framework.

7.4.2 Two Modes of executeInBackground()

`executeInBackground()` selects different backgrounding strategies based on hook type:

```
// hooks.ts L205-265
function executeInBackground({ asyncRewake, ... }): boolean {
  if (asyncRewake) {
    // Mode A: don't register to async hook registry, run in background directly
    // Don't call shellCommand.background() (avoids spillToDisk destroying in-memory stdout capture)
    void shellCommand.result.then(async result => {
      await new Promise(resolve => setImmediate(resolve))  // wait for I/O drain
      if (result.code === 2) {
        enqueuePendingNotification({ ... })  // wake model
      }
    })
    return true
  }

  // Mode B: standard async hook → register to registry
  if (!shellCommand.background(processId)) return false
  registerPendingAsyncHook({ processId, hookId, asyncResponse, ... })
  return true
}
```

Mode A (asyncRewake): Does not call `shellCommand.background()` because that method spills to disk when the process stdout exceeds the memory threshold, destroying the in-memory stdout capture that hooks expect. asyncRewake uses an independent memory buffer, only waking the model when exit code is 2.

Mode B (standard async): Calls `shellCommand.background()` to register with the async hook registry, which manages completion callbacks and timeouts for background processes.

7.4.3 Three-Way Race (Promise.race)

After backgrounding completes, the main flow waits for process termination, but `Promise.race` listens to three possible outcomes simultaneously:

```
// hooks.ts L1273-1277
const result = await Promise.race([
  childIsAsyncPromise,   // asyncResolve called when resolved (first line was async JSON)
  childClosePromise,     // resolved when process closed and streams drained (sync path)
  childErrorPromise,     // rejected when process 'error' event fires
])
```

- **childIsAsyncPromise wins**: If the hook declares async at runtime, the main flow immediately gets the result and continues; the hook finishes in the background
- **childClosePromise**: Normal path completion; hook finishes and gets full output
- **childErrorPromise**: Process error immediately throws; doesn't get stuck

This ensures the main flow never waits indefinitely, regardless of whether the hook executes synchronously or asynchronously.

7.5 Prompt Request Serialization

Command Hooks can send JSON-formatted prompt requests, which Claude Code responds to and writes back to stdin:

```
// Prompt request in hook stdout
{ "prompt": "request_id", "message": "...", "options": [...] }

// Claude Code response
{ "prompt_response": "request_id", "selected": "option_key" }
```

These prompt interaction lines are stripped from output before final stdout parsing, preventing contamination of results.

---

8. Prompt Hook Executor in Detail

8.1 Execution Flow

Defined in: `restored-src/src/utils/hooks/execPromptHook.ts`

```
1. addArgumentsToPrompt(hook.prompt, jsonInput)
   → Replace $ARGUMENTS with full JSON input string

2. createUserMessage({ content: processedPrompt })
   → Construct user message (Note: does NOT go through processUserInput, won't trigger UserPromptSubmit Hook!)

3. queryModelWithoutStreaming({
     messages: [...messages, userMessage],
     systemPrompt: "You are evaluating a hook...",
     outputFormat: { type: 'json_schema', schema: { ok, reason } },
     model: hook.model ?? getSmallFastModel(),  // Default Haiku
     signal: combinedSignal,
     ...
   })

4. extractTextContent(response.message.content) → parse text response

5. hookResponseSchema().safeParse(json)
   → Validate { ok: boolean, reason?: string }

6. Return HookResult
```

```
   ok=true  → outcome: 'success'
   ok=false → outcome: 'blocking' + blockingError
```

8.2 Key Features

Prompt Hook doesn't trigger recursion: `createUserMessage` directly creates a message object, bypassing `processUserInput`, avoiding infinite recursion of `UserPromptSubmit` Hooks.

The source comment (`execPromptHook.ts` L40-42) explicitly states:

> Create user message directly — no need for processUserInput which would trigger UserPromptSubmit hooks and cause infinite recursion.

Note

Haiku model is the default: `getSmallFastModel()` returns the fastest, lowest-cost model, minimizing Prompt Hook execution cost.

Error-tolerant principle: Prompt Hook takes a tolerant approach toward its own execution errors (JSON parse failure, Schema validation failure) — returns `non_blocking_error` without blocking the main flow; but when LLM returns `ok: false`, it triggers a `blocking` result, stopping the operation. This ensures hooks can safely experiment (errors don't affect the main flow) while having the authority to make critical decisions.

8.3 Compile-Time Type Consistency Assertion

```
// types/hooks.ts:196-200
import type { IsEqual } from 'type-fest'
type Assert<T extends true> = T
type _assertSDKTypesMatch = Assert<
  IsEqual<SchemaHookJSONOutput, HookJSONOutput>
>
```

This is a **type-level test**: using `type-fest`'s `IsEqual` utility, it **asserts at compile time** that the `SyncHookJSONOutput` type inferred from the Zod schema and the `HookJSONOutput` TypeScript type manually defined in the SDK are **exactly identical**. If the SDK and Zod schema diverge, compilation fails rather than waiting until runtime to discover the issue. This is critical for maintaining type consistency in large systems.

Fixed output Schema:

```
{
  type: 'object',
  properties: {
    ok: { type: 'boolean' },
    reason: { type: 'string' }
  },
  required: ['ok'],
  additionalProperties: false
}
```

---

9. Agent Hook Executor in Detail

9.1 Execution Flow

Defined in: `restored-src/src/utils/hooks/execAgentHook.ts`

```
1. addArgumentsToPrompt(hook.prompt, jsonInput)

2. Create StructuredOutputTool (forces agent to return structured result)

3. Filter toolset:
   - Remove all Agent-disallowed tools (ALL_AGENT_DISALLOWED_TOOLS)
   - Remove duplicate StructuredOutputTool
   - Keep all other tools (Agent can use full toolset)
   Note: Nested agent tools (like AgentTool) are forbidden, preventing hook agent escape

4. Register Session-level StructuredOutput enforcement Hook
   registerStructuredOutputEnforcement(setAppState, hookAgentId)
   → Add Stop event FunctionHook
   → Check if message history has structured_output tool call
   → If not, force-return error message after 5s timeout

5. Create agentToolUseContext:
   - agentId: hook-agent-<uuid>
   - alwaysAllowRules adds Read(<transcriptPath>) to allow transcript reading
   - mode: 'dontAsk' (no permission confirmation needed, no dialogs)

6. for await (message of query({...}))  multi-turn loop
   - Max 50 turns (MAX_AGENT_TURNS)
   - Look for messages with attachment.type === 'structured_output'

7. Parse structured_output → hookResponseSchema()
   ok=true  → outcome: 'success'
   ok=false → outcome: 'blocking' + blockingError

8. Cleanup: clearSessionHooks() + remove StructuredOutput enforcement Hook
```

Note

What ALL_AGENT_DISALLOWED_TOOLS prohibits and why

Tools that are filtered out and the reasons:

- `AgentTool` → forbids creating sub-agents (prevents infinite recursion)
- `ExitPlanModeV2Tool` / `EnterPlanModeTool` → forbids entering/exiting Plan Mode (prevents bypassing stop hook validation logic)
- `AskUserQuestionTool` → forbids asking the user questions (hook execution should be fully automated)
- `TaskStopTool` → forbids stopping tasks
- `WorkflowTool` → forbids recursive workflow execution

Core purpose: **prevent hook agent from escaping** — hook agents can only execute validation in a restricted environment, unable to interfere with the main agent loop

9.2 Mechanism for Enforcing StructuredOutput

Agent Hook registers a Session-level `FunctionHook` via `registerStructuredOutputEnforcement()`, ensuring the agent must call `SyntheticOutputTool` to end the conversation:

```
// hookHelpers.ts
registerStructuredOutputEnforcement(setAppState, hookAgentId)
// → Add type: 'function' hook to 'Stop' event
```

```
// → In Stop event, check if message history has structured_output tool call
// → If not, return error message forcing termination
```

This is a **dynamic AOP (Aspect-Oriented Programming)** pattern: aspects are dynamically registered when the agent hook starts and automatically removed after it ends, without modifying the agent's core logic.

Note

Core purpose of StructuredOutput enforcement

An agent running 50 turns might never call StructuredOutputTool — it reads files, judges conditions are met, and just finishes with no structured_output output. Without the Stop event enforcement check, this hook would wait until timeout and return cancelled, and the user would have no idea what happened.

So this is a **fallback contract**: the agent must explicitly call the StructuredOutput tool to output `{ok, reason}`, otherwise it's treated as failed.

9.2.1 Three-Tier Signal Cancellation Management

```
// execAgentHook.ts L78-82
const { signal: parentTimeoutSignal, cleanup: cleanupCombinedSignal } =
  createCombinedAbortSignal(signal, { timeoutMs: hookTimeoutMs })
const onParentTimeout = () => hookAbortController.abort()
parentTimeoutSignal.addEventListener('abort', onParentTimeout)
const combinedSignal = hookAbortController.signal
```

Three-tier signal architecture:

1. `signal` (caller-provided AbortSignal) → external cancellation
2. `parentTimeoutSignal` (hook-level timeout) → timeout cancellation
3. `combinedSignal` (actual signal passed to query()) → passed to subagent

When parent timeout or signal abort occurs, it automatically cascades to abort the hook agent's internal controller via `addEventListener('abort', onParentTimeout)`, ensuring the subagent correctly receives the abort signal.

9.2.2 Dynamic alwaysAllow Rule Injection

```
// execAgentHook.ts L137-153
getAppState() {
  const appState = toolUseContext.getAppState()
  const existingSessionRules = appState.toolPermissionContext.alwaysAllowRules.session ?? []
  return {
    ...appState,
    toolPermissionContext: {
      ...appState.toolPermissionContext,
      mode: 'dontAsk' as const,
      alwaysAllowRules: {
        ...appState.toolPermissionContext.alwaysAllowRules,
        session: [...existingSessionRules, `Read(/${transcriptPath})`],
      },
    },
  }
}
```

The hook agent's system prompt informs the agent it can read the transcript file; here, a session-level `alwaysAllow` rule is dynamically injected, allowing the hook agent to read its own transcript without manual user approval. This is a **refined practice of the principle of least privilege** — only granting the hook agent permissions it actually needs.

Note

Practical significance of hook agent reading conversation history

The hook agent's system prompt explicitly states "You can read this file to analyze the conversation history", meaning the hook can perform validation based on the **entire main session's conversation history**, not just the current tool call input. For example: a PostToolUse hook can read the transcript, compare previous Write operations with current test results, and judge whether the problem was actually fixed. This makes Agent hook's validation capability far exceed that of Prompt hook — the latter can only make judgments based on a single tool call's JSON.

Injecting `Read(/${transcriptPath})` into alwaysAllow is so this step is silently approved — the hook executes completely automatically without requiring user approval throughout.

9.3 Prompt Hook vs Agent Hook Comparison

Feature	Prompt Hook	Agent Hook
Interaction mode	Single-turn (one LLM request)	Multi-turn (up to 50 turns of tool calls)
Default model	Haiku	Haiku (configurable)
Tool calls	❌	✅ Full toolset
Timeout	30 seconds	60 seconds
Output schema	`{ ok, reason }`	`{ ok, reason }`
Execution cost	Low	High
Applicable scenarios	Simple conditional judgment	Complex validation (needs file reading/command execution)

---

10. HTTP Hook Executor in Detail

10.1 Execution Flow

Defined in: `restored-src/src/utils/hooks/execHttpHook.ts`

```
1. URL allowlist check
   policy.allowedUrls.some(p => urlMatchesPattern(hook.url, p))

2. Build merged AbortSignal (with timeout)
   createCombinedAbortSignal(signal, { timeoutMs })

3. Header construction + env var interpolation
   headers[name] = interpolateEnvVars(value, allowedEnvVars)
   → CR/LF/NUL sanitization

4. Proxy configuration
   - Sandbox mode: go through SandboxManager network proxy
   - Env var proxy (HTTP_PROXY): via configureGlobalAgents()
   - Both skip SSRF Guard (DNS handled by proxy)

5. axios.post(hook.url, jsonInput, {
     signal: combinedSignal,
     proxy: sandboxProxy ?? false,
```

```
    lookup: ssrfGuardedLookup,  // SSRF protection
    maxRedirects: 0,
    validateStatus: () => true,  // don't filter any status codes
  })

6. Parse response
   status 2xx  → ok: true
   other       → ok: false + statusCode

7. HTTP response body JSON validation (same flow as Command Hook)
```

Note

HTTP Hook's default timeout is **10 minutes** (600000ms), far longer than Command Hook (600s) and Agent Hook (60s). Because HTTP requests depend on external services with unpredictable latency, setting a long timeout is a reasonable tolerance strategy.

Additionally, in sandbox mode requests go through the SandboxManager network proxy, which **enforces domain allowlist** — even if a hook configures a legitimate URL, a domain intercepted by the proxy returns 403. This is sandbox-level URL allowlist enforcement; the hook layer checks again, forming double-layer protection.

10.2 Environment Variable Interpolation Algorithm

```
function interpolateEnvVars(
  value: string,
  allowedEnvVars: ReadonlySet<string>,
): string {
  return value.replace(
    /\$\{([A-Z_][A-Z0-9_]*)\}|\$([A-Z_][A-Z0-9_]*)/g,
    (_, braced, unbraced) => {
      const varName = braced ?? unbraced
      if (!allowedEnvVars.has(varName)) {
        // Not in allowlist → replace with empty string (safe default)
        return ''
      }
      return process.env[varName] ?? ''
    },
  )
}
```

Allowlist intersection rules (three-layer intersection):

```
Hook config has allowedEnvVars: ['TOKEN']              → only $TOKEN gets replaced
Policy (settings) has allowedEnvVars: ['SECRET']       → only $SECRET gets replaced
Both configured simultaneously                         → take intersection → only what's in both gets replaced
```

Intersection semantics ensure that even if a hook config leaks an env var, the policy allowlist can still provide a safety net. This is the **principle of least privilege** applied to HTTP hook env var interpolation.

10.2.1 Precise Semantics of CRLF Sanitization

```
// execHttpHook.ts L76-79
function sanitizeHeaderValue(value: string): string {
  // eslint-disable-next-line no-control-regex
  return value.replace(/[\r\n\x00]/g, '')  // strip CR/LF/NUL
}
```

A malicious env var value (e.g. `$TOKEN = "secret\r\nX-Evil-Header: injected"`) would inject extra HTTP headers after interpolation; this function performs post-interpolation defense, stripping all control characters. `\r\n` is the core weapon for HTTP header injection, and `\x00` (NUL) can corrupt C string processing.

Note

The order of defenses matters: interpolate first, then sanitize. If reversed, `\r\n` injected after sanitization would be interpolated in again by a subsequent hook and sanitized again. But writing it this way is clearer — any interpolated value goes through sanitization, regardless of whether it came from config or env var.

10.3 Precise Semantics of SSRF Protection

```
// execHttpHook.ts L216
lookup: sandboxProxy || envProxyActive ? undefined : ssrfGuardedLookup,
```

When using sandbox proxy or environment variable proxy (`HTTP_PROXY`), **SSRF check is skipped**. Reason: DNS resolution is done by the proxy server; checking whether the proxy's own IP is private makes no sense (proxy is usually publicly reachable). SSRF Guard (`ssrfGuardedLookup`) is only used for direct HTTP requests (not going through proxy), validating that resolved IPs don't belong to private/link-local addresses.

Blocked IP ranges:

- `10.0.0.0/8` — Private Class A
- `172.16.0.0/12` — Private Class B (172.16–31.x.x)
- `192.168.0.0/16` — Private Class C
- `127.0.0.1` — Loopback address
- `169.254.0.0/16` — Link-local (APIPA, auto-private IP)

```
// ssrfGuard.ts — validate IP range after DNS resolution
lookup: ssrfGuardedLookup  // replaces default dns.lookup

// Blocked IP ranges:
// - 10.0.0.0/8      (Private A)
// - 172.16.0.0/12  (Private B, 172.16–31.x.x)
// - 192.168.0.0/16 (Private C)
// - 127.0.0.1      (Loopback)
// - 169.254.0.0/16 (Link-local, APIPA)
```

Note

The real reason SSRF checks are skipped in proxy mode

The old documentation said "DNS resolution is done by the proxy." A more accurate explanation is: enterprise proxies typically run on private IPs (e.g. `10.0.0.1:3128`). If SSRF Guard still validated DNS resolution results when going through a proxy, it would actually be validating **the proxy server's own IP** — enterprise proxy IPs in the private range would be falsely judged as attack targets and blocked. So in proxy mode, SSRF checks are skipped entirely, with the proxy itself responsible for secure access control.

---

11. Callback and Function Hooks

11.1 Callback Hook

Defined in: `restored-src/src/types/hooks.ts` (lines 211–226)

A Callback Hook is a **TypeScript callback registered by the framework internally or by SDK plugins**; it cannot be configured via `settings.json` and is used for capabilities that cannot be expressed in config files:

```
type HookCallback = {
  type: 'callback'
  callback: (
    input: HookInput,
    toolUseID: string | null,
    abort: AbortSignal | undefined,
    hookIndex?: number,
    context?: HookCallbackContext,
  ) => Promise<HookJSONOutput>
  timeout?: number
  internal?: boolean  // internal hooks don't count toward tengu_run_hook metric
}
```

Typical uses:

- `sessionFileAccessHooks` — Session-level analysis tracking file access (internal)
- `attributionHooks` — Commit attribution analysis (internal)
- External hooks registered by SDK via `registerHooks()`

The key difference between Callback Hook and Function Hook: Callback Hook receives the full `HookInput` (including tool parameters), while Function Hook only receives message history.

11.2 Function Hook

Defined in: `restored-src/src/utils/hooks/sessionHooks.ts` (lines 24–31)

A Function Hook is a Session-scoped TypeScript callback **that cannot be persisted to settings.json**, because its `callback` is a function reference that cannot be serialized:

```
type FunctionHook = {
  type: 'function'
  id?: string              // Optional ID (used for removeFunctionHook)
  timeout?: number         // Default 5000ms
  callback: FunctionHookCallback
  errorMessage: string     // blockingError message on execution failure
  statusMessage?: string   // UI status text
}

type FunctionHookCallback = (
  messages: Message[],     // Conversation history
  signal?: AbortSignal,
) => boolean | Promise<boolean>  // true=approve, false=block
```

Execution: Return `true` → `outcome: 'success'`; return `false` → `outcome: 'blocking'`.

Note

Why Function/Callback Hooks are not deduplicated?
Command/Prompt/Agent/HTTP Hooks use deduplication keys like `"command\0if_condition"` — two hooks with identical config are merged into one. But `FunctionHook`'s callback is a memory function reference; two different FunctionHooks with identical logic still can't compare references, so **deduplication is intentionally skipped**: every `addFunctionHook` call registers independently. This is actually correct behavior — different registration points need independent triggers (e.g., `registerStructuredOutputEnforcement` and user-defined FunctionHooks shouldn't override each other).

Note

Function Hook and StructuredOutput Enforcement
`execAgentHook` uses `registerStructuredOutputEnforcement()`: when an Agent Hook starts, it dynamically registers a `type: 'function'` Stop event Hook that checks whether the conversation history contains a `structured_output` tool call. If not, it forces an error return after a 5-second timeout, ensuring the Agent Hook always returns a structured result (rather than plain text). This is a **runtime AOP** pattern — dynamically adding aspects during target function execution, automatically removed after execution ends.

---

12. Async Hook Mechanism and AsyncHookRegistry

12.1 Two Async Modes

Command Hook supports two async mechanisms:

Mode A: Config-declared Async

```
{
  "type": "command",
  "command": "python3 long_task.py",
  "async": true,
  "asyncTimeout": 60000
}
```

Mode B: Self-describing Async (runtime detection)
Hook indicates async by outputting `{"async": true, ...}` on the first line of stdout.

Mode C: asyncRewake (with model wake)

```
{
  "type": "command",
  "command": "python3 long_task.py",
  "asyncRewake": true
}
```

exitCode=2 → wakes model via `enqueuePendingNotification(wrapInSystemReminder(...), mode: 'task-notification')`.

12.2 AsyncHookRegistry Core Logic

Defined in: `restored-src/src/utils/hooks/AsyncHookRegistry.ts`

```
class AsyncHookRegistry {
  private pending = new Map<string, PendingAsyncHook>()

  // Register new async hook
  registerPendingAsyncHook({ processId, hookId, shellCommand, ... }) {
```

```
    const stopProgressInterval = startHookProgressInterval({
      hookId, hookName, hookEvent,
      getOutput: () => {
        const taskOutput = pending.get(processId)?.shellCommand?.taskOutput
        return {
          stdout: await taskOutput.getStdout(),
          stderr: taskOutput.getStderr(),
          output: stdout + stderr,
        }
      },
    })
    pending.set(processId, { ..., stopProgressInterval })
  }

  // Periodically poll all pending hooks' status
  async checkForAsyncHookResponses(): Promise<SyncHookJSONOutput[]> {
    // For each pending hook:
    // 1. Check shellCommand.status === 'completed'
    // 2. Read first line of stdout JSON
    // 3. Check for {"async": true} marker (outdated, actually sync output)
    // 4. Parse JSON → HookResult
    // 5. finalizeHook(hook, exitCode, outcome)
    // 6. Return response array
  }
}
```

12.3 Special Handling of SessionStart Async Hooks

```
// When SessionStart Hook completes async
if (hook.hookEvent === 'SessionStart') {
  sessionStartCompleted = true
}
// → triggers invalidateSessionEnvCache()
// → because SessionStart Hook may have written CLAUDE_ENV_FILE, needs to reload env
```

12.4 Cleanup and Finalization

```
// Called at session end: kill all incomplete async hooks
async function finalizePendingAsyncHooks(): Promise<void> {
  for (const hook of pendingHooks.values()) {
    if (hook.shellCommand?.status === 'completed') {
      await finalizeHook(hook, result.code, 'success')
    } else {
      hook.shellCommand?.kill()
      await finalizeHook(hook, 1, 'cancelled')
    }
  }
  pendingHooks.clear()
}
```

---

13. Session Hooks: Session-Scoped Hooks

13.1 SessionHooksState Data Structure

Defined in: `restored-src/src/utils/hooks/sessionHooks.ts`

```
// Using Map instead of plain object — .set()/.delete() don't change the container reference
// This lets store.ts's Object.is(next, prev) short-circuit, avoiding listener triggers
type SessionHooksState = Map<string, SessionStore>

type SessionStore = {
  hooks: {
    [event in HookEvent]?: SessionHookMatcher[]
  }
}

type SessionHookMatcher = {
  matcher: string
  skillRoot?: string
  hooks: Array<{
    hook: HookCommand | FunctionHook
    onHookSuccess?: OnHookSuccess  // Callback after hook succeeds
  }>
}
```

13.2 Core APIs

```
// Add Session Hook (persistently registered to session)
addSessionHook(setAppState, sessionId, event, matcher, hook, onHookSuccess?, skillRoot?)
// Typical caller: Agent frontmatter hooks (registered at each agent start, cleaned up at session end)

// Add Function Hook (returns hookId)
addFunctionHook(
  setAppState, sessionId, event, matcher,
  callback: (messages, signal) => boolean | Promise<boolean>,
  errorMessage: string,
  options?: { timeout?: number, id?: string }
): string
// Typical caller: registerStructuredOutputEnforcement in execAgentHook

// Remove Function Hook (precise deletion by ID)
removeFunctionHook(setAppState, sessionId, event, hookId)

// Read (returns hooks grouped by event)
getSessionHooks(appState, sessionId, event?): Map<HookEvent, SessionDerivedHookMatcher[]>
getSessionFunctionHooks(appState, sessionId, event?): Map<HookEvent, FunctionHookMatcher[]>
// Without event param, returns hooks for all events

// Cleanup (called at session end, kills all pending async hooks)
```

```
clearSessionHooks(setAppState, sessionId)
```

13.3 Deduplication Mechanism

Defined in: `restored-src/src/utils/hooks.ts` (`getMatchingHooks` function)

Hooks need deduplication before execution to prevent the same Hook from being registered multiple times (e.g., multiple plugins define the same Hook):

```
// Deduplication key construction rule
function hookDedupKey(m: MatchedHook, payload: string): string {
  return `${m.pluginRoot ?? m.skillRoot ?? ''}\0${payload}`
}

// Command Hook dedup key = "shell\0command\0if_condition"
// Prompt Hook dedup key = "prompt\0if_condition"
// Agent Hook dedup key = "prompt\0if_condition"
// HTTP Hook dedup key = "url\0if_condition"
// Function/Callback Hook → no dedup (each is unique)

// Special case: settings file deduplication
// When userSettings and projectSettings point to the same file (running from $HOME)
// resolve() the path and deduplicate with Set
const seenFiles = new Set<string>()
```

---

14. Plugin Hooks and Hot Reload

14.1 Plugin Hook Loading Flow

Defined in: `restored-src/src/utils/plugins/loadPluginHooks.ts`

```
1. Get list of all enabled plugins
2. For each plugin:
   a. Scan .ts files in hooks/ directory → compile → register
   b. Read hooks field from package.json → register directly
3. Add source: 'pluginHook' and pluginName metadata to each Hook
```

14.2 Special Variables for Plugin Hooks

Plugin Hook shell commands can access additional template variables:

Variable	Description
`${CLAUDE_PLUGIN_ROOT}`	Plugin root directory
`${CLAUDE_PLUGIN_DATA}`	Plugin data directory
`${user_config.<key>}`	Plugin user config (declared in manifest)
`CLAUDE_PLUGIN_OPTION_<KEY>`	Plugin options as environment variables

14.3 Hot Reload Mechanism

```
// Watch plugin directory changes, real-time reload hooks
function setupPluginHookHotReload(): void {
  watchPluginDirs(async (changedPlugin) => {
    // 1. Reload hooks of changedPlugin
    const hooks = await loadPluginHooks()
    // 2. Update Hook registry
    await updateHookRegistry(hooks)
  })
}
```

Hot reload targets **individual plugins** rather than all plugins: `watchPluginDirs` detects which plugin directory changed, and only reloads that plugin's hooks, avoiding the performance cost of full reloads. After reload, the old hooks in the registry are atomically replaced via `updateHookRegistry`.

Note

Hot reload boundary: config-style Hooks vs. registered Hooks
Hot reload can only update hooks registered via plugin directory `.ts` files or `package.json`. For config-style hooks in `settings.json` (Command/Prompt/Agent/HTTP Hooks), hot reload doesn't apply — changes to these are handled separately by `settings.json`'s file watcher (triggering `updateHooksConfigSnapshot()`).
Hot reload is snapshot-driven, not just file watching
Hot reload is triggered by changes to 4 fields in `getPluginAffectingSettingsSnapshot()`: `enabledPlugins`, `extraKnownMarketplaces`, `strictKnownMarketplaces`, `blockedMarketplaces`. `loadAllPluginsCacheOnly()`'s memoized result depends on these 4 fields, so even if a user only changes a remotely managed blocklist (without touching enabledPlugins), hot reload is still triggered — the snapshot diffed, the memoize cache invalidated, and plugin hooks were reloaded.

14.4 Cleanup When Plugin is Disabled

When a plugin is uninstalled, `pruneRemovedPluginHooks()` removes all hooks registered by that plugin, preventing orphaned hooks from remaining.

---

15. Matcher System and Conditional Filtering

15.1 Matcher Concept

> **A Matcher is a Hook's "routing label" — determining under what conditions a set of Hooks is triggered.**

Each event's hooks are grouped by the `matcher` field; a matcher is a pattern to match `matchQuery`:

```
// Configured in settings.json
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",           // Match tool name "Bash"
        "hooks": [{ type: "command", command: "check_bash.sh" }]
      },
      {
        "matcher": "Read|Write",     // OR match
        "hooks": [{ type: "prompt", prompt: "..." }]
      },
      {
```

```
        "matcher": ".*",             // All tools (wildcard)
        "hooks": [{ type: "http", url: "https://..." }]
      }
    ]
  }
}
```

How it works: When an event triggers, the system extracts `matchQuery` from `hookInput` (e.g., `tool_name` for `PreToolUse`), then iterates through all Matchers and uses `matchesPattern()` to judge whether there's a match. All hooks under successfully matched Matchers are added to the execution queue.

Note

Matchers determine "which tools' hooks can run"; `if` conditions determine "which specific parameter combinations within these tools run". Two-level filtering — coarse first, precise second — avoids making a large number of hooks make useless execution judgments.

15.2 matchesPattern() Algorithm

> **This is the core matching function for Matchers — judges whether an event context (tool name/file name) matches the pattern string in config.**

Defined in: `restored-src/src/utils/hooks.ts` (lines 1346–1381)

```
function matchesPattern(matchQuery: string, matcher: string): boolean {
  // Case 1: Empty or wildcard
  if (!matcher || matcher === '*') return true

  // Case 2: Simple string or pipe-separated (no regex special chars)
  if (/^[a-zA-Z0-9_|]+$/.test(matcher)) {
    if (matcher.includes('|')) {
      // OR exact match: 'Read|Write' → ['Read', 'Write']
      const patterns = matcher.split('|').map(p => normalizeLegacyToolName(p.trim()))
      return patterns.includes(matchQuery)  // compare after normalization
    }
    return matchQuery === normalizeLegacyToolName(matcher)
  }

  // Case 3: Regular expression
  try {
    const regex = new RegExp(matcher)
    if (regex.test(matchQuery)) return true
    // Also check legacy names (for backward compatibility with old tool names)
    for (const legacyName of getLegacyToolNames(matchQuery)) {
      if (regex.test(legacyName)) return true
    }
    return false
  } catch {
    return false  // Conservative fallback on invalid regex
  }
}
```

Note

Match priority: pipe OR > exact match > regex
The system first checks whether the matcher is a "simple string without regex special chars" — this is the actual format of most configs (`Bash`, `Edit|Write`). Regex is only attempted when the simple format doesn't match. This avoids regex performance overhead (compilation + backtracking) for simple match scenarios. Invalid regex (e.g. `*foo`) conservatively degrades to non-match rather than erroring — prevents user typos from causing all hooks to fail.

15.3 `if` Conditional Filtering

The `if` field uses Permission rule syntax for finer-grained filtering of Hooks:

```
PreToolUse: matcher="Bash", if="git *"
→ Execute only when Bash command matches "git *"

PreToolUse: matcher="Write", if="package.json"
→ Execute only when Write target filename contains "package.json"
```

`if` condition evaluation is implemented by `prepareIfConditionMatcher()`; for Bash tools it uses `tree-sitter` to parse the command structure before matching, and for other tools it does string matching.

Note

tree-sitter parsing vs. string matching: safe command filtering
For Bash tools, `if` conditions use tree-sitter (rather than simple string `includes`) to parse command structure before matching. This prevents users from constructing bypass methods with special characters — for example, `git; rm -rf /` won't be incorrectly judged as matching `git *`. For Write/Read/Edit and other file-type tools, string matching is done directly on file paths in `tool_input` (because tool input is already structured JSON, no parsing needed).

15.4 Hook Priority Ordering

Defined in: `restored-src/src/utils/hooks/hooksSettings.ts` (`sortMatchersByPriority`)

```
// SOURCES order determines priority (lower array index = higher priority)
const SOURCES = ['userSettings', 'projectSettings', 'localSettings', 'pluginHook', 'builtinHook']

// pluginHook and builtinHook have lowest priority (999)
// At same priority, sort by matcher name alphabetically
```

---

16. Hook Execution Event Broadcasting System

16.1 Three Types of Execution Events

Defined in: `restored-src/src/utils/hooks/hookEvents.ts`

```
// 1. Started event
type HookStartedEvent = {
  type: 'started'
  hookId: string
  hookName: string
  hookEvent: string
}

// 2. Progress event (streaming stdout/stderr push)
type HookProgressEvent = {
```

```
  type: 'progress'
  hookId: string
  hookName: string
  hookEvent: string
  stdout: string
  stderr: string
  output: string
}

// 3. Response event (final result)
type HookResponseEvent = {
  type: 'response'
  hookId: string
  hookName: string
  hookEvent: string
  output: string
  stdout: string
  stderr: string
  exitCode?: number
  outcome: 'success' | 'error' | 'cancelled'
}
```

16.2 Event Filtering Mechanism

```
function shouldEmit(hookEvent: string): boolean {
  // SessionStart / Setup always broadcast (backward compatibility)
  if (ALWAYS_EMITTED_HOOK_EVENTS.includes(hookEvent)) return true
  // Other events require allHookEventsEnabled=true (includeHookEvents SDK option)
  return allHookEventsEnabled && HOOK_EVENTS.includes(hookEvent)
}
```

Note

"Always broadcast" event design
`SessionStart` and `Setup` are always broadcast regardless of whether the SDK has registered an event handler. This isn't because they're especially important, but because: these two events are the most user-perceptible hook lifecycle nodes (session start and environment initialization), and it's a serious product experience problem if users configure them but see no feedback. Making them "always broadcast" ensures users see logs in any SDK usage pattern — even if the SDK developer forgot to set `includeHookEvents: true`.

16.3 Pending Event Buffer

If the Handler hasn't registered yet, events go into a buffer (max 100):

```
function emit(event: HookExecutionEvent): void {
  if (eventHandler) {
    eventHandler(event)  // Send directly
  } else {
    pendingEvents.push(event)  // Enqueue
    if (pendingEvents.length > MAX_PENDING_EVENTS) {
      pendingEvents.shift()  // Evict oldest if over capacity
    }
  }
}
```

16.4 Progress Push Implementation

> **During hook execution, subprocess output is polled every 1 second; real-time stdout/stderr is pushed to frontend/SDK via event broadcasting, which consumers use to render a real-time log panel.**

Core mechanism:

- **`startHookProgressInterval()`**: Timed polling + throttling. Internally uses `setInterval` to call `getOutput()` every N milliseconds (default 1s) to get the subprocess's current output, comparing with `lastEmittedOutput` — if output hasn't changed, skip the push to avoid duplicate events.
- **`interval.unref()`**: Key detail — lets the timer not block process exit. Even if a Hook is still running, when Claude Code's main process exits, the timer doesn't prevent the process from truly terminating.
- **Event broadcast path**: `emit()` in `hookEvents.ts` → registered `eventHandler` → SDK layer converts to `HookProgressEvent` messages → frontend renders real-time logs (see 16.1 Three Types of Execution Events).
- **Silent protection**: `shouldEmit()` checks the `allHookEventsEnabled` flag — when hook events aren't enabled (e.g., normal CLI runs), Progress events are silently discarded without affecting the main flow.

```
function startHookProgressInterval(params: {
  hookId, hookName, hookEvent,
  getOutput: () => Promise<{ stdout, stderr, output }>,
  intervalMs?: number,  // default 1000ms
}): () => void {
  const interval = setInterval(async () => {
    const { stdout, stderr, output } = await params.getOutput()
    if (output === lastEmittedOutput) return  // throttle: skip if output unchanged
    lastEmittedOutput = output
    emitHookProgress({ hookId, hookName, hookEvent, stdout, stderr, output })
  }, params.intervalMs ?? 1000)
  interval.unref()  // don't block process exit
  return () => clearInterval(interval)  // return stop function
}
```

---

17. Key Algorithm Details

17.1 URL Wildcard Matching

```
function urlMatchesPattern(url: string, pattern: string): boolean {
  const escaped = pattern.replace(/[.+?^${}()|[\]\\]/g, '\\$&')
  const regexStr = escaped.replace(/\*/g, '.*')
  return new RegExp(`^${regexStr}$`).test(url)
}
```

Principle: Convert the wildcard `*` in the allowlist to regex's `.*`, then do strict full-string matching from start to end.

17.2 Windows POSIX Path Conversion

```
// windowsPathToPosixPath() — pure JS regex, LRU-500 cache
// C:\Users\foo       → /c/Users/foo
// C:/Users/foo       → /c/Users/foo
// \\server\share     → //server/share (UNC preserved)
```

```
// Note: PowerShell paths are NOT converted
```

17.3 Shell Selection Algorithm

```
const shellType = hook.shell ?? DEFAULT_HOOK_SHELL
const isPowerShell = shellType === 'powershell'

if (isPowerShell) {
  child = spawn(pwshPath, ['-NoProfile', '-NonInteractive', '-Command', cmd], {...})
} else {
  const shell = isWindows ? findGitBashPath() : true  // Windows requires Git Bash
  child = spawn(cmd, [], { shell, ... })
}
```

PowerShell skips:

- Windows POSIX path conversion
- `.sh` auto prepend `bash`
- `CLAUDE_CODE_SHELL_PREFIX`

17.4 Prompt Request Detection and Serialization

Command Hooks support sending prompt requests to Claude Code during execution:

```
// Detect prompt request in stdout
const validation = promptRequestSchema().safeParse(parsed)
if (validation.success) {
  // Serialize and handle response
  promptChain = promptChain.then(async () => {
    const response = await reqPrompt(promptReq)
    child.stdin.write(jsonStringify(response) + '\n')
  })
  processedPromptLines.add(trimmed)  // mark for later removal from stdout
}
```

17.5 Multi-Agent Session Isolation

Hook session isolation is achieved via `agentId`:

```
// In executeHooks
const sessionId = toolUseContext?.agentId ?? getSessionId()

// In getHooksConfig
const registeredHooks = getRegisteredHooks()?.[hookEvent]
// SDK callbacks: grouped by agentId
// Plugin hooks: globally shared (pluginRoot namespace isolation)
// Session hooks: isolated by sessionId
```

Note

Precise meaning of isolation: main thread agent and subagent see different hooks
Main thread (main session): `sessionId = getSessionId()`; Subagent (working thread started by `Agent` tool): `sessionId = toolUseContext.agentId` (e.g. `hook-agent-<uuid>` or custom ID passed by agent). This means:

- Subagent's hooks don't affect main thread's hook config
- Two Subagents with different `agentId`s each see independent Session Hooks
- Plugin hooks are globally shared (isolated by `pluginRoot` namespace), not isolated by agent — this is intentional design, as plugins are globally loaded infrastructure

---

18. Security Mechanisms

18.1 Multi-Layer Security Overview

Layer	Mechanism	Location	Description
L1	Workspace trust check	`shouldSkipHookDueToTrust()`	All hooks require interactive trust
L2	ManagedOnly strategy	`shouldAllowManagedHooksOnly()`	In managed scenarios, forbid user-defined hooks
L3	Completely disable hooks	`shouldDisableAllHooksIncludingManaged()`	Admin can completely disable
L4	HTTP URL allowlist	`execHttpHook.ts`	Block non-allowlist URLs
L5	SSRF DNS protection	`ssrfGuard.ts`	Block private IP requests
L6	Header injection protection	`sanitizeHeaderValue()`	Filter CR/LF/NUL
L7	Env var allowlist	`interpolateEnvVars()`	Prevent sensitive info leakage
L8	Shell timeout protection	`TOOL_HOOK_EXECUTION_TIMEOUT_MS`	10-minute forced termination
L9	AbortSignal cascade	`createCombinedAbortSignal()`	External cancellation signal propagation

18.2 Trust Level Enumeration

```
enum TrustLevel {
  untrusted = 0,    // Don't execute any hooks
  trusted = 1,      // Only execute user-level hooks
  explicit = 2,     // Execute all hooks (including project config)
}
```

Trust level is determined via interactive dialog on first startup (`claude --trust <dir>` can skip). The level determines the strictness of hook execution scope: in untrusted mode all hooks are skipped; in trusted mode only user-level hooks in `~/.claude/settings.json` execute; in explicit mode hooks from all config locations (including project directory `.claude/settings.json`) execute.

Note

The essence of trust levels is "who is responsible for hooks"

- `untrusted`: User hasn't decided to trust this project yet; executing any hook is risky, skip all
- `trusted`: User trusts their own `$HOME` personal config but not necessarily team/project hooks
- `explicit`: User explicitly chose to trust project config, willing to execute any hooks declared in the project

This is similar to the Unix filesystem permission model: files in your own home directory are usually trusted (trusted), while shared project directories from others need explicit authorization (explicit).

18.3 Plugin Hook Special Handling

Plugin Hooks are **excluded** in `managedOnly` mode:

```
// In getHooksConfig
if (managedOnly && 'pluginRoot' in matcher) {
  continue  // skip Plugin Hook
}
```

This ensures plugins cannot bypass security policies through hooks in managed environments.

---

19. Configuration System and Priority

19.1 Configuration Files and Sources

Source	File Path	Priority	Description
User Settings	`~/.claude/settings.json`	Highest	User global config
Project Settings	`.claude/settings.json`	Second	Project-level config
Local Settings	`.claude/settings.local.json`	Third	Local override
Policy Settings	Managed policy config	Fourth	Enterprise managed
Plugin Hooks	`~/.claude/plugins/*/hooks/`	Lowest	Plugin directories
Session Hooks	In-memory Map	Temporary	Cleared at session end
Frontmatter	Code comments	Temporary	Agent-level

Note

Actual meaning of priority: within the same event + same matcher, later sources override earlier ones
The "priority" in the table is the override order, not execution order. Actually, when multiple sources define hooks for the same event match, smaller index in the `SOURCES` array means higher priority (`localSettings > projectSettings > userSettings`), and `pluginHook` and `builtinHook` have lowest priority. This means: if both userSettings and projectSettings define the same event's hook, both execute (merged), not the latter overriding the former. Only within the same source does the "last definition wins" semantics truly apply.

19.2 Configuration Merging Rules

```
function getHooksFromAllowedSources(): HooksSettings {
  const policySettings = getSettingsForSource('policySettings')

  // 1. Admin completely disabled
  if (policySettings?.disableAllHooks === true) return {}

  // 2. Managed-only mode
  if (policySettings?.allowManagedHooksOnly === true) {
    return policySettings.hooks ?? {}
  }

  // 3. Plugin-isolated mode: forbid user/project/local hooks
  if (isRestrictedToPluginOnly('hooks')) {
    return policySettings?.hooks ?? {}
  }

  // 4. User disabled all hooks: only keep managed hooks
  if (mergedSettings.disableAllHooks === true) {
    return policySettings?.hooks ?? {}
  }

  // 5. Normal mode: merge all sources
  return mergedSettings.hooks ?? {}
}
```

19.3 Same-File Deduplication

When running from `$HOME` directory, `~/.claude/settings.json` is both User Settings and Project Settings. Deduplicated via `Set` after `resolve()`:

```
const seenFiles = new Set<string>()
for (const source of sources) {
  const filePath = getSettingsFilePathForSource(source)
  if (filePath) {
    const resolvedPath = resolve(filePath)
    if (seenFiles.has(resolvedPath)) continue
    seenFiles.add(resolvedPath)
  }
}
```

---

20. Snapshot Mechanism

20.1 Why Snapshots Are Needed

Hook config is captured as a snapshot **at application startup** (`captureHooksConfigSnapshot()`), not read from disk on each execution. Reasons:

1. **Performance**: Avoids filesystem access on every hook execution
2. **Consistency**: Ensures hook config remains unchanged within a session
3. **Security**: Captured **before** the trust dialog is shown (no trust state exists yet), but trust checks happen **at execution time**

20.2 Snapshot Lifecycle

```
At startup
  captureHooksConfigSnapshot()
  → Read all config files
  → Apply ManagedOnly policy filtering
  → Store in initialHooksConfig global variable

On config change
  updateHooksConfigSnapshot()
  → resetSettingsCache()  // Clear settings cache first (prevent stale disk data)
  → Re-read and update snapshot

On query
  getHooksConfigFromSnapshot()
  → If snapshot empty, call captureHooksConfigSnapshot()
```

```
  → Return snapshot (or null)
```

20.3 Relationship with Settings Cache

```
// hooksConfigSnapshot.ts:104-111
function updateHooksConfigSnapshot(): void {
  // Reset the session cache to ensure we read fresh settings from disk.
  // Without this, the snapshot could use stale cached settings when the user
  // edits settings.json externally and then runs /hooks - the session cache
  // may not have been invalidated yet (e.g., if the file watcher's stability
  // threshold hasn't elapsed).
  resetSettingsCache()
  initialHooksConfig = getHooksFromAllowedSources()
}
```

This is an **active defense** design: file watchers have a stability threshold (preventing churn from frequent changes). If a user saves settings.json from an external editor, when the snapshot updates it must actively clear the cache, otherwise it might read stale data.

Test isolation: `resetHooksConfigSnapshot()` also calls `resetSdkInitState()`, ensuring tests don't contaminate each other through residual SDK initialization state.

---

21. UI Layer Integration

21.1 `/hooks` CLI Command

Defined in: `restored-src/src/commands/hooks/hooks.tsx`

Provides an interactive interface for managing Hooks:

Subcommand	Function
`/hooks list`	List all hooks (grouped by source)
`/hooks add`	Add new hook (interactive guide)
`/hooks remove <id>`	Remove hook
`/hooks enable <id>`	Enable hook
`/hooks disable <id>`	Disable hook
`/hooks run <id>`	Manually trigger hook (for testing)
`/hooks edit <id>`	Edit hook config
`/hooks logs <id>`	View hook execution logs

21.2 HooksConfigMenu Component

Defined in: `restored-src/src/components/hooks/HooksConfigMenu.tsx`

React component displaying all Hooks grouped by source:

```
User Settings
  ├─ PreToolUse:Bash   echo "running bash"
  └─ PostToolUse:Write update_fts.sh

Project Settings
  └─ PreToolUse:*      log_all_tools.sh

Plugin Hooks
  └─ PreToolUse:Read   plugin-x/read_checker.sh

Session Hooks (temporary)
  └─ PreToolUse:Write  (in-memory)
```

21.3 UI Event Flow

```
HookStartedEvent     → Spinner displays "Running hook: xxx..."
HookProgressEvent    → Real-time stdout/stderr panel update
HookResponseEvent    → Result card (success / error / blocked) display
AggregatedHookResult→ Main UI updates tool execution result or blocking prompt
```

---

22. End-to-End Data Flow Diagram

```
                User Submits Prompt
                       │
                       ▼
            ┌──────────────────────┐
            │  Agent Loop /        │
            │  ToolCoordinator     │
            └──────────────────────┘
                       │
                       ▼
            ┌──────────────────────────────┐
            │ executeHooks({               │
            │   hookInput: PreToolUseHookInput,  │
            │   toolUseID: 'tool-123',     │
            │   signal: AbortSignal,       │
            │   toolUseContext: {...},     │
            │   messages: [...],           │
            │ })                           │
            └──────────────────────────────┘
                       │ AsyncGenerator
                       ▼
            ┌──────────────────────────────┐
            │ Guard Checks                 │
            │ ① shouldDisableAllHooksIncluding... │ → early return
            │ ② CLAUDE_CODE_SIMPLE         │ → early return
            │ ③ shouldSkipHookDueToTrust()  │ → early return
            └──────────────────────────────┘
                       │
                       ▼
            ┌──────────────────────────────┐
            │ getMatchingHooks()           │
```

```
snapshot → getHooksConfig()
registered → getRegistered()
session → getSessionHooks()
dedup by command+shell+if
filter by if condition
```

```
Fast path?
All callbacks → direct exec (-70%)
```

```
hookPromises[] parallel launch
One async function*() per hook
```

```
Callback Hook
executeHook
Callback()
→ JSON Output
```

```
Prompt Hook
execPromptHook
→ queryModel()
→ Haiku LLM
→ {ok, reason}
```

```
Agent Hook
execAgentHook
→ query()
→ multi-turn
→ StructuredOut
```

```
execCommandHook() / execHttpHook()
spawn subprocess / axios.post
stdin.write(jsonInput + '\n')
stdout streaming → progress
first-line JSON detect async
executeInBackground() / wait
```

```
parseHookOutput(stdout)
validateHookJson() → Zod validation
processHookJSONOutput() → HookResult
```

```
Hook Execution Event Broadcast
emitHookStarted()
HookProgressEvent (every 1s)
emitHookResponse()
```

```
AggregatedHookResult
outcome=blocking → stop main flow
outcome=success → continue tool exec
outcome=non_blocking_error → log
outcome=cancelled → silent handle
```

---

23. System-Level Design Principles (Meta-Analysis)

After reading the entire Hook system source code, several **architectural design principles** that pervade the entire system can be distilled:

Principle 1: Defense in Depth
Security-related checks are distributed across multiple layers: trust check is done centrally at the entry, URL allowlist at the HTTP layer, SSRF at the DNS layer, header injection after interpolation. A single check's oversight doesn't compromise overall security.

Principle 2: Performance Adapts to Scenario
The system doesn't pursue optimal single execution path; instead, it detects scenario characteristics at runtime (all internal callbacks? Does the batch need JSON serialization?), dynamically selecting the fast path or full path. This avoids "all hooks paying the cost of the fastest hook."

Principle 3: Snapshot Isolation Instead of Real-time Reading
By capturing a config snapshot at startup, it achieves a balance between config immutability and performance. Changes are triggered via explicit `updateHooksConfigSnapshot()`, ensuring predictability.

Principle 4: Type Guards as Anti-Corruption Layer
`syncHookResponseSchema` / `hookJSONOutputSchema` and other Zod validation layers isolate untrusted hook output (JSON) from system internal types. Validation failures are wrapped as `non_blocking_error` rather than exceptions, embodying a pragmatic philosophy of "trust but verify."

Principle 5: Test Isolation as First-Class Citizen
`resetHooksConfigSnapshot()` also resets SDK initialization state, `resetSettingsCache()` ensures no cache pollution between tests. This demonstrates that the system designer gave full consideration to production-level testability.

---

24. Appendix: Design Pattern Summary

Pattern 1: Strategy Pattern

Different hook types are selected by the unified execution interface within `executeHooks`'s dispatch logic:

```
if (hook.type === 'callback')    → executeHookCallback(...)
if (hook.type === 'function')    → executeFunctionHook(...)
if (hook.type === 'prompt')      → execPromptHook(...)
if (hook.type === 'agent')       → execAgentHook(...)
if (hook.type === 'http')        → execHttpHook(...)
```

```
if (hook.type === 'command')     → execCommandHook(...)
```

Each executor accepts the same signature and returns `Promise<HookResult>`.

Pattern 2: Observer Pattern

The event bus in `hookEvents.ts` implements publish-subscribe:

```
let eventHandler: HookEventHandler | null = null
const pendingEvents: HookExecutionEvent[] = []

function emit(event: HookExecutionEvent): void {
  if (eventHandler) {
    eventHandler(event)  // Already subscribed: dispatch immediately
  } else {
    pendingEvents.push(event)  // Not subscribed: enqueue buffer
  }
}
```

Pattern 3: Registry Pattern

`AsyncHookRegistry` and `SessionHooksState` use in-memory Maps as registries:

```
const pendingHooks = new Map<string, PendingAsyncHook>()
// O(1) add/remove/query

type SessionHooksState = Map<string, SessionStore>
// agent concurrent registration: .set() doesn't trigger listeners (Object.is short-circuits)
```

Supplementary: Map vs Record Performance Trap

In `parallel()` high-concurrency workflows, `SessionHooksState` using `Map` instead of `Record` is critical:

Approach	Per-registration complexity	N agents total complexity	Store listener triggers
`Record` + spread	O(N) object copy	$O(N^2)$	~30 listeners per trigger
`Map`	O(1)	O(N)	Zero (Object.is short-circuits)

The source comment (`sessionHooks.ts` L48-62) explicitly documents this design decision. This is a **targeted optimization after identifying the high-frequency write path** — the difference is imperceptible in normal usage but significant in enterprise high-concurrency workflows.

Pattern 4: Snapshot Pattern

`HooksConfigSnapshot` captures a config snapshot at startup; subsequent reads go directly from memory:

```
Application startup → captureHooksConfigSnapshot() → initialHooksConfig = {...}
Hook execution → getHooksConfigFromSnapshot() → read directly from memory
Config change → updateHooksConfigSnapshot() → recapture snapshot
```

Pattern 5: Pipeline / Generator Pattern

`executeHooks` is an AsyncGenerator, dividing hook execution into multiple stages with step-by-step yield:

```
async function* executeHooks({ ... }) {
  yield { message: { type: 'progress', ... } }  // notify start
  for (const hookPromise of hookPromises) {
    yield* hookPromise  // spread each hook's results
  }
  // final aggregated result
}
```

Callers consume results one by one via `for await (const result of executeHooks(...))`.

Pattern 6: Decorator Pattern

`createCombinedAbortSignal` combines multiple signal sources:

```
const { signal, cleanup } = createCombinedAbortSignal(
  signal,           // External AbortSignal
  { timeoutMs }     // Timeout AbortSignal
)
// combinedSignal triggers when any source triggers
// cleanup() clears the timer
```

Pattern 7: Anti-Corruption Layer Pattern

Hook output JSON must pass Zod validation before being used by the system; non-compliant output is wrapped as `hook_non_blocking_error`, never polluting the main flow:

```
const validation = hookJSONOutputSchema().safeParse(parsed)
if (!validation.success) {
  return {
    outcome: 'non_blocking_error',
    message: createAttachmentMessage({ type: 'hook_non_blocking_error', ... })
  }
}
```

---

25. File Index

Here is a detailed index of all 19 core files in the Hook system:

#	File Path	Core Exports	Key Types
1	`utils/hooks.ts`	`executeHooks()`, `getMatchingHooks()`, `execCommandHook()`, `processHookJSONOutput()`, `HookResult`, `HookBlockingError`	~2700 lines, core engine
2	`utils/hooks/AsyncHookRegistry.ts`	`registerPendingAsyncHook()`, `checkForAsyncHookResponses()`, `finalizePendingAsyncHooks()`, `PendingAsyncHook`	Async hook global registry
3	`utils/hooks/execHttpHook.ts`	`execHttpHook()`, `urlMatchesPattern()`, `interpolateEnvVars()`, `sanitizeHeaderValue()`, `getSandboxProxyConfig()`	HTTP webhook executor

#	File Path	Core Exports	Key Types
4	`utils/hooks/execPromptHook.ts`	`execPromptHook()`	Prompt → Haiku LLM executor
5	`utils/hooks/execAgentHook.ts`	`execAgentHook()`	Agent → Subagent multi-turn executor
6	`utils/hooks/hookEvents.ts`	`emitHookStarted()`, `emitHookProgress()`, `startHookProgressInterval()`, `emitHookResponse()`, `setAllHookEventsEnabled()`, `HookStartedEvent/ProgressEvent/ResponseEvent`	Event broadcasting system
7	`utils/hooks/hooksSettings.ts`	`getAllHooks()`, `getHooksForEvent()`, `sortMatchersByPriority()`, `isHookEqual()`, `getHookDisplayText()`	Settings read + priority
8	`utils/hooks/hooksConfigManager.ts`	Hook config CRUD	Config management layer
9	`utils/hooks/hooksConfigSnapshot.ts`	`captureHooksConfigSnapshot()`, `updateHooksConfigSnapshot()`, `getHooksConfigFromSnapshot()`, `shouldAllowManagedHooksOnly()`, `shouldDisableAllHooksIncludingManaged()`	Snapshot + managed policy
10	`utils/hooks/sessionHooks.ts`	`addSessionHook()`, `addFunctionHook()`, `removeFunctionHook()`, `getSessionHooks()`, `clearSessionHooks()`, `SessionHooksState`, `FunctionHook`	Session-scoped hooks
11	`utils/hooks/hookHelpers.ts`	`addArgumentsToPrompt()`, `createStructuredOutputTool()`, `hookResponseSchema()`, `registerStructuredOutputEnforcement()`	Helper functions
12	`types/hooks.ts`	`HookCallback`, `HookCallbackMatcher`, `HookResult`, `AggregatedHookResult`, `HookBlockingError`, `FunctionHook`, `hookJSONOutputSchema()`, `syncHookResponseSchema()`, `promptRequestSchema()`	Core type definitions + Zod Schema
13	`schemas/hooks.ts`	Zod validation schema for hook config	Config validation
14	`utils/plugins/loadPluginHooks.ts`	`loadPluginHooks()`, `setupPluginHookHotReload()`, `pruneRemovedPluginHooks()`	Plugin hook loading
15	`utils/hooks/registerFrontmatterHooks.ts`	Register hooks from doc comments	Frontmatter registration
16	`utils/hooks/registerSkillHooks.ts`	Register hooks from Skill definition files	Skill registration
17	`components/hooks/HooksConfigMenu.tsx`	Hook config React menu component	UI component
18	`commands/hooks/hooks.tsx`	`/hooks` CLI command	CLI command
19	`entrypoints/sdk/coreTypes.ts`	`HOOK_EVENTS` constant array, `HookEvent` type	SDK core types